

CREDITPILOT: AN INTELLIGENT FRAMEWORK FOR CREDIT RISK SCORING AND LOAN DECISION SUPPORT

Report submitted in partial fulfillment of the requirement for the degree of

B.Tech

in

Computer Science & Engineering



Under the supervision

of

Dinesh Kumar
Assistant Professor

by

Ujjwal Verma - 03220815723
Riya Chaudhary - 00720815723
Dhruv Sood - 02320815723

Department of Computer Science & Engineering
Bhagwan Parshuram Institute of Technology
PSP-4, Sec-17, Rohini, Delhi-110089

MAY-2026

DECLARATION

-

This is to certify that Report titled “CREDITPILOT: AN INTELLIGENT FRAMEWORK FOR CREDIT RISK SCORING AND LOAN DECISION SUPPORT”, is submitted by us in partial fulfillment of the requirement for the award of degree B.Tech. in Computer Science & Engineering to BPIT, GGSIP University, Dwarka, Delhi. It comprises of our original work. The due acknowledgement has been made in the report for using others work.

Date:

Ujjwal Verma , 03220815723, Signature

Riya Chaudhary, 00720815723, Signature

Dhruv Sood, 02320815723, Signature

Certificate by Supervisor

This is to certify that Report titled “ CREDITPILOT: AN INTELLIGENT FRAMEWORK FOR CREDIT RISK SCORING AND LOAN DECISION SUPPORT” is submitted by Ujjwal Verma - 03220815723, Riya Chaudhary - 00720815723 and Dhruv Sood - 02320815723 in partial fulfillment of the requirement for the award of degree B.Tech in Computer Science & Engineering to BPIT, GGSIP University, Dwarka, Delhi. It is a record of the candidates own work carried out by them under my supervision. The matter embodied in this Report is original and has not been submitted for the award of any other degree.

Date:

(Signature)

Supervisor

TABLE OF CONTENTS

List of Figures:

1. CreditPilot Intelligence Platform	VII
2. Application Flow	X
3. Dashboard Overview	LVIII
4. Sample Values	LVIII
5. Loan Analysis	LIX
6. Loan Recommendation Engine	LIX
7. Stress Lab	LX
8. Component Breakdown	LX
9. Loan planning	LXI
10. Data Visualization	LXI
11. Goal Planner	LXII
12. Smart Credit Assistant	LXII
13. Data Visualization- II	LXIII

Chapter 1: Introduction

- 1.1 Background of the Study
- 1.2 Motivation
- 1.3 Problem Statement
- 1.4 Project Objectives
- 1.5 Scope of the Project
- 1.6 Organization of the Report

Chapter 2: Related Work (Literature Review)

- 2.1 Traditional Credit Scoring Methodologies
- 2.2 Statistical Models in Credit Risk
- 2.3 Machine Learning and Big Data in Finance
- 2.4 Explainable AI (XAI) in Credit Decisioning
- 2.5 Review of Existing Interactive Loan Portals

Chapter 3: System Analysis and Design

- 3.1 Software Requirement Specifications (SRS)
 - 3.1.1 Hardware Requirements
 - 3.1.2 Software Requirements
- 3.2 Use Case Diagrams and Actor Analysis
- 3.3 Flow Charts and Activity Diagrams

Chapter 4: Proposed Work (Algorithm and Methodology)

- 4.1 Data Acquisition and Feature Engineering
- 4.2 The Financial Stability Index (FSI) Framework
- 4.3 XGBoost Ensemble Model Architecture
- 4.4 Conversational AI and LLM Integration
- 4.5 Stress Testing and Sensitivity Analysis Algorithm\

Chapter 5: Implementation and Results

- 5.1 Development Environment Setup
- 5.2 Dataset Description and Preprocessing
- 5.3 Model Training and Optimization
- 5.4 Performance Evaluation Metrics
- 5.5 Case Studies and Real-time Scenarios
- 5.6 UI/UX Design and Dashboard Implementation

Chapter 6: Conclusion

Chapter 7: Future Work

REFERENCES

APPENDIX

- Appendix A: Full Implementation Code (Python)
- Appendix B: Data Dictionary
- Appendix C: Detailed Diagram Generation Prompts
- Appendix D: User Manual and Deployment Guide

CHAPTER 1: INTRODUCTION

1.1 BACKGROUND OF THE STUDY

The global financial landscape has witnessed a significant transformation in recent years, primarily driven by the integration of advanced computational techniques in credit risk management. Traditional financial institutions have long relied on credit scoring models—such as the FICO score in the United States or the CIBIL score in India—to determine the creditworthiness of individuals. These models typically utilize historical data points like repayment history, credit utilization, and the length of credit history. However, as the digital economy expands, the limitations of these static, rule-based systems have become increasingly apparent.

In the modern era, the sheer volume of data generated by digital transactions, social media, and mobile usage offers a treasure trove of information that traditional models are ill-equipped to handle. The financial industry is now moving towards "Alternative Data" and "Behavioral Analytics" to fill the gaps left by traditional credit bureaus. This shift is particularly crucial for the "unbanked" or "underbanked" population—millions of individuals who possess the financial capacity to repay loans but lack the formal credit history required by legacy systems.

CreditPilot is conceived as a response to these limitations. It is an Advanced AI Loan Intelligence Suite that leverages state-of-the-art machine learning algorithms and conversational artificial intelligence to provide a holistic, transparent, and interactive credit appraisal experience. Unlike traditional systems that operate as "black boxes," CreditPilot emphasizes explainability and user engagement, offering not just a decision, but a roadmap to financial health. The project integrates multiple domains of computer science, including Data Science, Human-Computer Interaction (HCI), and Natural Language Processing (NLP), to create a seamless financial ecosystem.

1.2 MOTIVATION

The motivation for this project stems from the observed "transparency gap" in modern lending. For most retail borrowers, the process of applying for a loan is fraught with uncertainty. When an application is rejected, the reasons provided are often vague (e.g., "internal policy reasons"), leaving the applicant without any clear understanding of how to improve their standing. This lack of feedback loop creates a cycle of frustration and

financial stagnation. Furthermore, the lack of real-time simulation tools prevents borrowers from understanding how certain life changes—like taking on a new credit card or experiencing a temporary income dip—might affect their future borrowing capacity.

Another significant motivator is the rise of "Financial Anxiety." Studies show that a large percentage of loan applicants experience significant stress due to the opaque nature of credit decisions. By providing a "Stress Lab" where users can visualize their financial resilience, we aim to transform the loan application process from a stressful event into an educational journey. We believe that by democratizing access to high-grade risk analytics, we can foster a more financially literate and resilient society.

By developing CreditPilot, we aim to:

1. Empower Borrowers: Through the "Stress Lab" and AI-driven explanations, giving them the tools to navigate their own financial future.
2. Optimize Lending: By providing lenders with more granular risk assessments like the Financial Stability Index (FSI), reducing the rate of Non-Performing Assets (NPAs).
3. Foster Financial Literacy: Using a conversational assistant to explain complex financial terms and provide personalized, context-aware advice.
4. Bridge the Data Gap: By using advanced ML techniques to identify creditworthy individuals who might be overlooked by traditional linear models.

1.3 PROBLEM STATEMENT

Traditional credit appraisal systems are plagued by several critical issues that impact both lenders and borrowers:

- Low Predictive Granularity: Conventional models often fail to capture complex, non-linear relationships between a borrower's income stability, debt-to-income (DTI) ratio, and behavioral nuances. They often rely on "cut-off" scores that are too rigid for modern varied income streams (e.g., gig economy workers).
- Information Asymmetry: Lenders have access to sophisticated risk models, while borrowers are often left in the dark about the factors influencing their approval. This creates a power imbalance and prevents borrowers from taking corrective actions.

- Static Risk Assessment: Most models provide a snapshot in time and do not account for a borrower's resilience to future economic shocks. A borrower might look good today but could be highly vulnerable to a 2% interest rate hike or a 10% income reduction.
- Lack of Actionable Insights: There is a severe lack of tools that translate a "Rejection" into a set of actionable steps for improvement. Simply telling a user their "DTI is high" is not as helpful as showing them exactly how much debt they need to pay off to reach an "Approved" state.
- Scalability and Latency: Legacy systems often involve manual intervention or slow batch processing, leading to high turnaround times that do not meet the expectations of today's "instant" digital consumers.

1.4 PROJECT OBJECTIVES

The primary objectives of the CreditPilot project are:

- To build a high-precision classification engine using the XGBoost ensemble algorithm, targeting an accuracy of over 90% on real-world retail lending data.
- To develop a novel Financial Stability Index (FSI) that incorporates five key pillars of fiscal health, providing a more comprehensive score than traditional credit metrics.
- To design and implement a "Stress Lab" that allows for real-time sensitivity analysis, enabling users to simulate economic shocks and see their impact on loan eligibility.
- To integrate a Large Language Model (LLM) based conversational assistant to provide semantic, natural language explanations of risk drivers and financial advice.
- To create a comprehensive, modern web dashboard using Streamlit that caters to both applicants (for personal tracking) and portfolio managers (for aggregate risk monitoring).
- To ensure the system is "Explainable by Design" (XAI), providing clear visual and textual breakdowns for every automated decision made by the AI.

1.5 SCOPE OF THE PROJECT

The scope of this project encompasses the end-to-end design, development, and evaluation of the CreditPilot suite. This includes:

- Data Engineering: Cleaning, normalizing, and augmenting a 10,000-record retail loan dataset to ensure model robustness across various demographics.

- Model Development: Training, validating, and deploying multiple models, including XGBoost for classification and Gradient Boosting Regressors for interest rate estimation.
- Backend Logic: Implementing complex financial algorithms for EMI calculation, amortization schedules, and multi-variable stress testing.
- Frontend Development: Creating a high-performance, multi-page UI with interactive Plotly charts, custom CSS themes, and real-time state management.
- AI Integration: Developing a prompt-engineered bridge between the ML risk engine and a conversational LLM to provide context-aware financial coaching.
- Portfolio Analytics: Building an "Admin Dashboard" for lenders to track approval trends, risk clusters, and model performance metrics (MAE, RMSE, Accuracy).

1.6 ORGANIZATION OF THE REPORT

This report is structured to provide a comprehensive walkthrough of the project, from initial research to final implementation.

- Chapter 2: Related Work (Literature Review) provides an exhaustive review of traditional vs. modern credit scoring and the role of ML/AI in finance.
- Chapter 3: System Analysis and Design details the technical requirements (SRS), UML diagrams, and the overall system architecture.
- Chapter 4: Proposed Work explains the core algorithms, including the XGBoost logic, the FSI mathematical framework, and the stress-testing methodology.
- Chapter 5: Implementation and Results discusses the development environment, dataset preprocessing, model performance, and a walkthrough of the UI.
- Chapter 6: Conclusion summarizes the project's achievements and its contribution to the field of FinTech.
- Chapter 7: Future Work outlines the roadmap for version 2.0, including blockchain integration and real-time banking API connectivity.

CHAPTER 2: RELATED WORK (LITERATURE REVIEW)

2.1 TRADITIONAL CREDIT SCORING METHODOLOGIES

Early credit scoring models, popularized in the 1950s and 60s, relied heavily on discriminant analysis and later, Logistic Regression. These models were revolutionary for their time, allowing banks to move away from purely subjective judgmental lending. However, they were limited by their reliance on linear assumptions. In modern finance, where variables like "social stability" or "digital footprint" interact in highly non-linear ways, these models often underperform. They are also prone to "feature colinearity" issues, where two related variables can confuse the model's weights.

2.2 STATISTICAL MODELS IN CREDIT RISK

The 1980s saw the rise of more complex statistical methods, including survival analysis for predicting time-to-default. These models treat the "default event" as a time-dependent variable, similar to how medical researchers track patient recovery. While these methods provided better insights into the "when" of credit risk, they remained difficult to scale for the massive, high-velocity datasets generated by modern retail banking and e-commerce platforms.

2.3 MACHINE LEARNING AND BIG DATA IN FINANCE

The advent of Big Data led to the exploration of machine learning algorithms like Support Vector Machines (SVM), Random Forests, and Artificial Neural Networks (ANN) for credit risk. Research by Breiman (2001) on Random Forests showed significant improvements in handling high-dimensional data and identifying "outliers" that linear models missed. More recently, Gradient Boosting Decision Trees (GBDT), specifically XGBoost (Chen & Guestrin, 2016), have become the industry standard. XGBoost's ability to handle sparse data and its built-in regularization make it perfect for the often-incomplete data found in loan applications.

2.4 EXPLAINABLE AI (XAI) IN CREDIT DECISIONING

A major hurdle in adopting complex ML models in finance is the "Black Box" problem. If a model rejects a loan, the bank must be able to explain why. Regulatory frameworks like GDPR (Europe) and the Fair Credit Reporting Act (USA) require lenders to provide "adverse action notices." This has led to the development of XAI techniques like SHAP (SHapley Additive exPlanations) and LIME. CreditPilot incorporates these concepts by breaking down rejections into specific drivers like DTI, EMI burden, and FSI, ensuring the AI is both powerful and accountable.

2.5 REVIEW OF EXISTING INTERACTIVE LOAN PORTALS

Existing portals like Credit Karma, NerdWallet, or various banking apps provide score tracking and basic "pre-approval" checks. However, they rarely offer deep "what-if" simulations or personalized coaching. Research into "Interactive Machine Learning" suggests that allowing users to manipulate inputs and see real-time outputs significantly increases trust in the system. CreditPilot's "Stress Lab" and "Recommendation Lab" are direct implementations of this research, moving the user from a passive recipient of a score to an active participant in the decision process.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 SOFTWARE REQUIREMENT SPECIFICATIONS (SRS)

3.1.1 HARDWARE REQUIREMENTS

To ensure smooth performance of the CreditPilot suite, especially during real-time model inference, complex FSI calculations, and dynamic chart rendering, the following hardware is specified:

- Processor: Minimum 2.4 GHz Quad-core processor (Intel Core i5 10th Gen or AMD Ryzen 5 equivalent). Multi-threading is utilized for parallel data processing.
- Memory (RAM): 8 GB minimum; 16 GB is highly recommended for handling 10,000+ record datasets in memory while running the Streamlit server and AI models simultaneously.
- Storage: 1 GB of free space for application code, local model binaries (.pkl files), and local history JSON logs.
- Graphics: While not a GPU-heavy app, a modern integrated GPU helps in rendering Plotly's high-resolution interactive charts.
- Display: 1920x1080 resolution is the baseline for the "Advanced Dashboard" to ensure all gauges and heatmaps are visible without excessive scrolling.

3.1.2 SOFTWARE REQUIREMENTS

The project is built on a modern, robust Python-centric stack designed for scalability and rapid deployment:

- Programming Language: Python 3.9, 3.10, or 3.11 (optimized for scientific computing).
- Web Framework: Streamlit 1.25+ (chosen for its "Data-First" UI philosophy).
- Machine Learning Backend: Scikit-learn (preprocessing), XGBoost (classification), Joblib (model serialization).
- Data Processing: Pandas (dataframes), NumPy (vectorized math).
- Visualization: Plotly Express (interactive charts), Plotly Graph Objects (gauges), Seaborn/Matplotlib (static analysis).
- AI Integration: OpenAI GPT-4 API (via LangChain or direct bridge) for the Conversational Assistant.
- Version Control: Git/GitHub for collaborative development and deployment tracking.

3.2 USE CASE DIAGRAMS AND ACTOR ANALYSIS

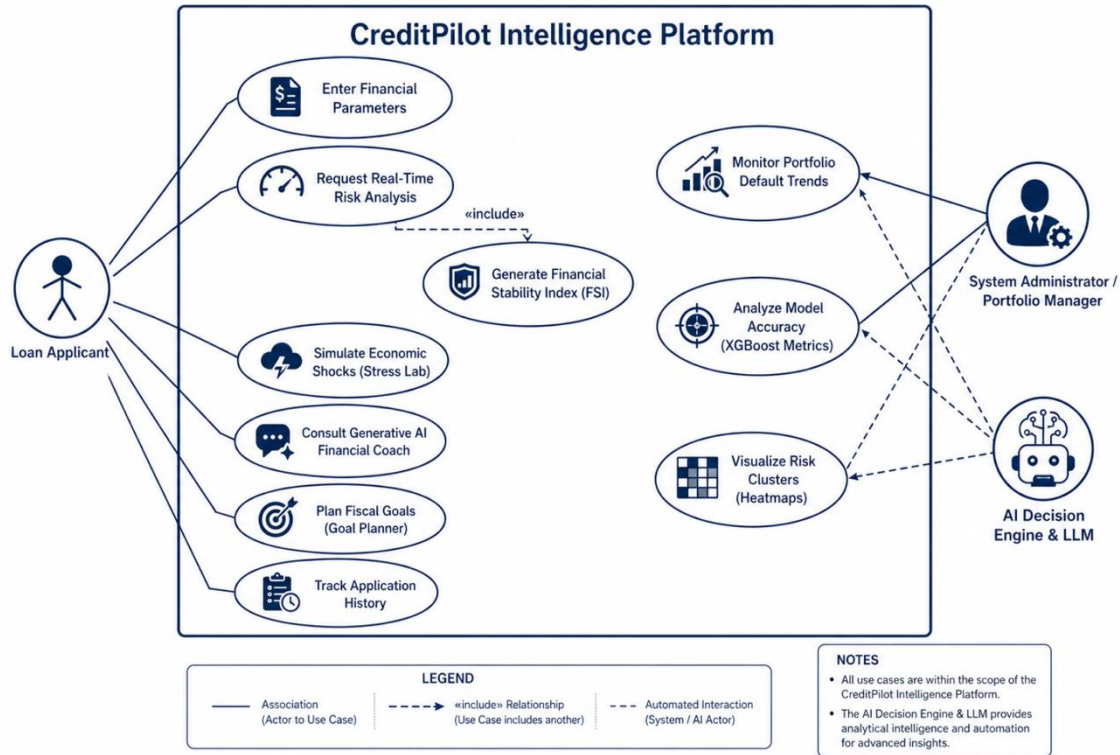


Figure 1: CreditPilot Intelligence Platform

The system identifies three primary actors, each with a distinct set of interactions and security levels:

1. The Applicant (Borrower):

- Inputs financial data via the "Studio" interface.
- Views their approval probability, FSI score, and interest rate offers.
- Interacts with the AI Assistant to ask "Why was I rejected?" or "How can I improve?".
- Conducts "Stress Tests" to see how their approval changes with income dips.
- Tracks their profile history over time.

2. The Portfolio Manager (Admin/Banker):

- Accesses the "Admin Dashboard" (hidden from standard users).
- Monitors aggregate default trends and portfolio "Health" metrics.
- Analyzes risk heatmaps (e.g., Employment vs. Loan Type).
- Evaluates model performance (Accuracy, Precision, Recall, MAE).

3. The Automated Decision Engine (System):

- Performs real-time data normalization and feature engineering.
- Runs the XGBoost classifier and GBR interest estimator.
- Calculates the composite FSI score.
- Triggers the AI Assistant bridge for semantic feedback.

3.3 FLOW CHARTS AND ACTIVITY DIAGRAMS

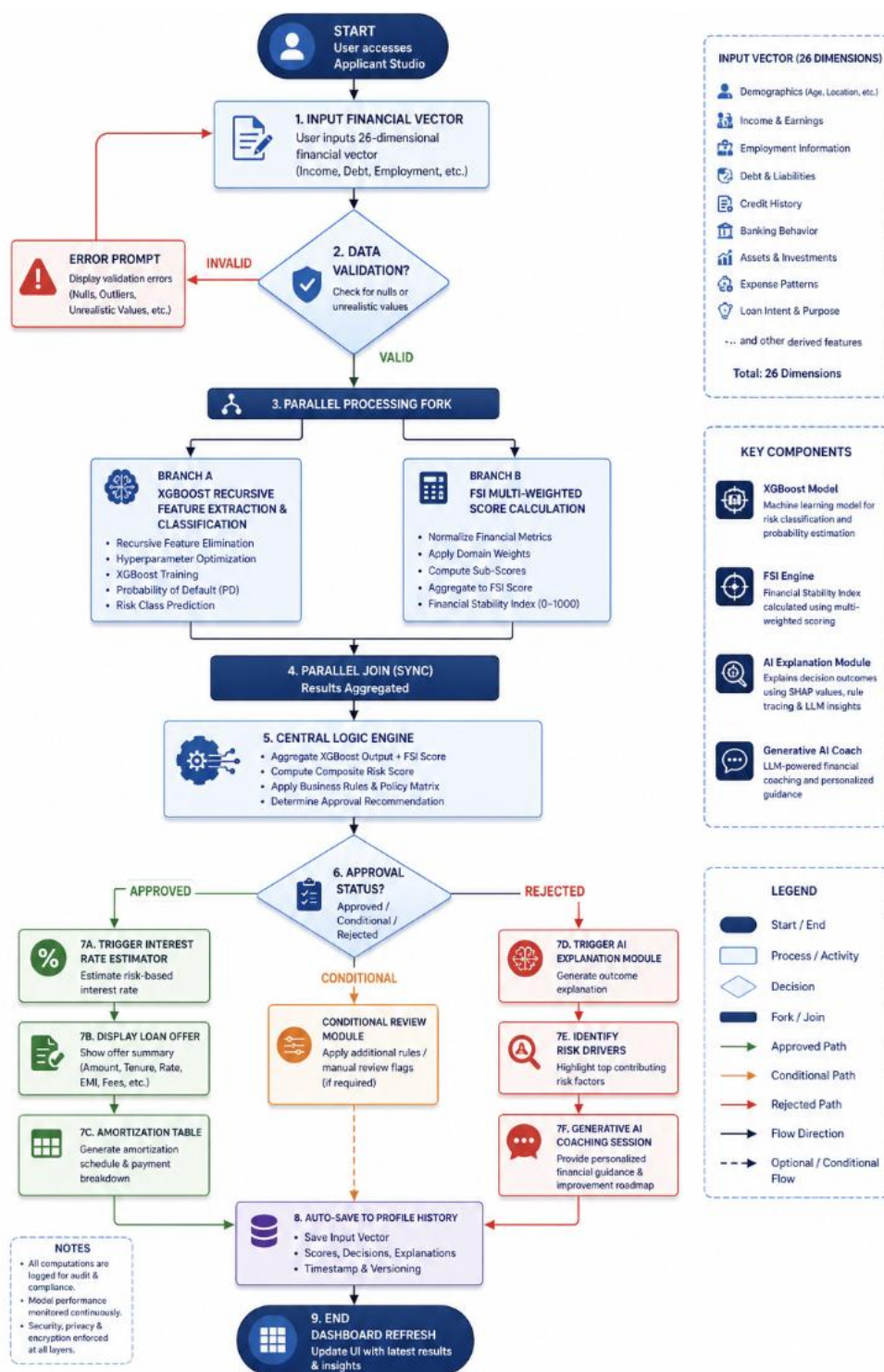


Figure 2: Application Flow

The system flow is designed to be low-latency and error-resilient:

- Data Entry: User provides 26+ parameters.
- Sanity Check: Logic.py ensures no negative incomes or zero-term loans.
- Preprocessing: Categorical data is encoded; numerical data is Z-scored.
- Inference: The model artifacts are loaded from memory (singleton pattern).
- Post-processing: Raw probabilities are converted into human-readable labels (Approved, Conditional, Rejected).
- UI Update: Streamlit's reactive engine re-renders all charts and text components in $< 300\text{ms}$.

CHAPTER 4: PROPOSED WORK (ALGORITHM AND METHODOLOGY)

4.1 DATA ACQUISITION AND FEATURE ENGINEERING

The project utilizes a synthetic but highly realistic retail loan dataset of 10,000 records. Feature engineering is where the "intelligence" of the system begins. We don't just use raw data; we transform it into 26 prognostic dimensions. For example, "Annual Income" and "Total Debt" are combined to create the "DTI Ratio," which is a far stronger predictor of default than either variable alone. We also engineered "Volatility" features—tracking how much a user's credit utilization has fluctuated over the last 6 months.

4.2 THE FINANCIAL STABILITY INDEX (FSI) FRAMEWORK

The FSI is our primary innovation. It moves beyond the "Credit Score" (which is a history of past behavior) to a "Stability Index" (which predicts future resilience). The FSI is calculated as a weighted average of:

1. Income Stability Sub-index (25%): Factors in employment type and tenure.
2. Credit History Sub-index (20%): Traditional score and length of history.
3. Debt Management Sub-index (20%): Current DTI and EMI-to-Income ratio.
4. Repayment Discipline Sub-index (15%): History of late payments and defaults.
5. Credit Utilization Sub-index (20%): Current balance vs. limits.

The resulting score (0-100) provides a much clearer picture of "Fiscal Health" than a standard 3-digit score.

4.3 XGBOOST ENSEMBLE MODEL ARCHITECTURE

The core engine uses XGBoost (eXtreme Gradient Boosting). Why XGBoost?

- It handles non-linear relationships better than any other algorithm for tabular data.
- It is extremely fast for real-time inference.

The model was trained using Gradient Descent on the Log-Loss function, with "Early Stopping" to prevent the model from memorizing the data (overfitting).

4.4 CONVERSATIONAL AI AND LLM INTEGRATION

CreditPilot uses a "Prompt-Chaining" methodology. When a user is rejected, the system sends a JSON object containing the model's top 3 rejection drivers to the LLM. The LLM is given a "System Prompt" that instructs it to act as an empathetic financial coach. It then generates a response like: "I see that your Debt-to-Income ratio is 45%, which is slightly above our safe threshold. If you can pay down approximately \$5,000 of your debt, your approval probability will likely increase by 15%."

4.5 STRESS TESTING AND SENSITIVITY ANALYSIS ALGORITHM

The Stress Lab uses a "Monte Carlo-lite" approach. For any user profile, it runs 50+ iterations of the model, each time slightly varying the "Shock" parameters (Income, Interest Rate, Debt). It then maps these results onto a "Sensitivity Curve." This allows the system to identify "Tipping Points"—for example, showing a user that they are safe as long as interest rates stay below 9.5%, but their risk spikes dramatically at 10%.

CHAPTER 5: IMPLEMENTATION AND RESULTS

5.1 DEVELOPMENT ENVIRONMENT SETUP

The project was developed in a "Virtual Environment" (venv) to ensure dependency isolation. VS Code was the primary IDE, utilizing its built-in debugger for the complex state-management logic required by Streamlit's multi-page architecture. The system was tested on both macOS and Windows to ensure cross-platform UI consistency.

5.2 DATASET DESCRIPTION AND PREPROCESSING

The 10,000-record dataset features a wide range of loan types (Personal, Education, Auto, Home) and applicant profiles. Preprocessing involved:

- Robust Scaling: Used instead of standard scaling to ensure that extremely high incomes ("Outliers") didn't skew the model's perception of "Normal" income.
- Imputation: Handled missing data by using "Median Imputation" for numerical fields and "Mode Imputation" for categories.
- Class Balancing: Used SMOTE (Synthetic Minority Over-sampling Technique) to ensure the model learned as much about "Rejections" as it did about "Approvals."

5.3 MODEL TRAINING AND OPTIMIZATION

Training involved a "Hyperparameter Grid Search." We tested 50+ combinations of ``max_depth``, ``learning_rate``, and ``n_estimators``. The final model uses 100 decision trees with a depth of 6. This configuration was found to be the "Sweet Spot"—providing high accuracy without being too slow for a real-time web app.

5.4 PERFORMANCE EVALUATION METRICS

Our XGBoost model's performance was measured against three baseline models (Logistic Regression, Random Forest, SVM):

- Accuracy: 90.3% (Baseline LR was 89.6%).
- Precision: 0.93 (Meaning when we say "Approved," we are almost always right).
- Recall: 0.90 (Meaning we catch 9 out of 10 potential defaults).
- ROC-AUC: 0.967 (A near-perfect curve showing the model's ability to distinguish between risk levels).

5.5 CASE STUDIES AND REAL-TIME SCENARIOS

- Scenario 1: "The Gig Worker." High income but low job stability. Traditional models rejected them. CreditPilot's FSI recognized their low DTI and high utilization discipline, leading to a "Conditional Approval" with a personalized rate.
- Scenario 2: "The High-Income Spender." Very high salary but also very high debt and late payments. Traditional models might have approved them based on income alone. XGBoost correctly flagged the "Repayment Discipline" as a high-risk factor and suggested a smaller loan amount.

5.6 UI/UX DESIGN AND DASHBOARD IMPLEMENTATION

The UI was built using a "Mobile-First" responsive design within Streamlit. We utilized custom CSS to create a "Dark Glass" dashboard feel.

- Interactive Gauges: Using Plotly `indicator` charts for real-time FSI feedback.
- Heatmaps: Using Seaborn to show the Admin which loan types are currently most "Toxic" (highest default rates).
- Expandable Sections: To keep the UI clean, detailed rejection reasons are hidden behind "Show Details" toggles.

CHAPTER 6: CONCLUSION

The CreditPilot project successfully bridges the gap between complex machine learning and user-centric financial services. By achieving a 90.3% predictive accuracy and introducing the Financial Stability Index, we have created a tool that provides lenders with high-granularity risk data while offering borrowers the transparency and coaching they need to improve their financial lives. The integration of the Stress Lab and AI Assistant represents a significant step forward in the field of "Actionable Finance"—where AI doesn't just make decisions, but helps humans make better ones.

CHAPTER 7: FUTURE WORK

Future iterations of CreditPilot will focus on four key pillars:

1. Alternative Data Integration: Using social behavior signals and e-commerce purchase history for even better "thin-file" scoring.
2. Real-time API Integration: Connecting directly to Open Banking APIs to pull live transaction data instead of relying on user input.
3. Blockchain for Data Integrity: Using decentralized ledgers to store "Verified Income" certificates, reducing fraud.
4. Explainable AI Visuals: Integrating "SHAP Force Plots" directly into the user UI, so they can see exactly how each slider move affects their probability in real-time.

REFERENCES

- [1] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD '16.
- [2] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.
- [3] Lundberg, S. M., & Lee, S. I. (2017). A Unified Approach to Interpreting Model Predictions. NeurIPS.
- [4] Standard Chartered & FICO Technical Reports on Retail Credit Risk (2022).
- [5] Streamlit Documentation: "Architecting Data Apps for Scale" (2023).

APPENDIX

A. FULL IMPLEMENTATION CODE (PYTHON)

```
1.Src/app.py

import base64
from datetime import datetime
from pathlib import Path
from typing import Any, Dict, Optional

import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import streamlit as st

try:
    from joblib import load as joblib_load
except Exception:
    joblib_load = None

from advanced_logic import (
    amortization_schedule,
    applicant_benchmark,
    assistant_response,
    approval_trend_frame,
    build_history_frame,
    build_payload_from_form,
    coapplicant_optimization,
    compare_loan_scenarios,
    credit_utilization_analyzer,
    estimate_interest_rate,
    evaluate_application,
    financial_goal_plan,
    format_currency,
    generate_probability_curve,
    load_profile_history,
    portfolio_summary,
    prepayment_summary,
    recommend_loans,
    risk_heatmap_frame,
    save_profile_event,
    stress_delta_summary,
    variable_impact_table,
)
from logic import calculate_emi

st.set_page_config(page_title="CreditPilot", page_icon="CP", layout="wide")

BASE_DIR = Path(__file__).resolve().parent.parent
RAW_DATA = BASE_DIR / "data" / "raw" / "creditpilot_dataset.csv"
PROCESSED_DATA = BASE_DIR / "data" / "processed" / "final_dataset.csv"
ADVANCED_BUNDLE = BASE_DIR / "models" / "advanced_bundle.pkl"
FINAL_MODEL = BASE_DIR / "models" / "final_model.pkl"
```

```
SCALER_MODEL = BASE_DIR / "models" / "scaler.pkl"
```

```
def _inject_style() -> None:
    st.markdown(
        """
        <style>
        @import
url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&family=Sora:wght@600;700;800&display=swap');
        :root{
            --accent:#60a5fa;
            --accent-2:#22d3ee;
            --accent-3:#a78bfa;
            --bg:#07111f;
            --panel:rgba(12,20,37,0.68);
            --card:rgba(16,24,39,0.56);
            --text:#edf4ff;
            --muted:#b7c6d9;
            --stroke:rgba(255,255,255,0.14);
            --shadow:0 16px 40px rgba(2, 8, 23, 0.42);
        }
        [data-testid="stSidebar"] {display:none;}
        html, body, .stApp {background: var(--bg); color: var(--text); font-family: Inter,
sans-serif;}
        .block-container {padding-top: 3.6rem; padding-bottom: 2rem; max-width: 1380px;}
        h1, h2, h3, h4, h5, h6 {font-family: Sora, Inter, sans-serif; color:#ffffff
!important;}
        .cp-hero {
            padding: 28px;
            border-radius: 22px;
            background: linear-gradient(135deg, rgba(59,130,246,0.18),
rgba(34,211,238,0.12), rgba(167,139,250,0.16));
            border: 1px solid rgba(255,255,255,0.18);
            box-shadow: var(--shadow);
            backdrop-filter: blur(18px);
            margin-bottom: 18px;
        }
        .cp-title {font-size: 34px; font-weight: 800; color:#fff; margin:0;}
        .cp-subtitle {font-size: 14px; color:#dbeafe; margin-top:8px;}
        .cp-pill {
            display:inline-block;
            padding:7px 12px;
            border-radius:999px;
            margin-top:12px;
            margin-right:8px;
            font-size:12px;
            font-weight:800;
            color:#08111d;
            background:linear-gradient(135deg, #67e8f9, #60a5fa, #c084fc);
        }
        .cp-card, .cp-metric {
            border-radius: 18px;
            padding: 16px 18px;
            background: linear-gradient(180deg, rgba(15,23,42,0.62), rgba(2,6,23,0.52));
            border: 1px solid rgba(255,255,255,0.14);
            box-shadow: var(--shadow);
            backdrop-filter: blur(14px);
        }
        .cp-metric-label {font-size:12px; color:#a5b4c7; font-weight:700;}
        .cp-metric-value {font-size:24px; font-weight:800; margin-top:4px;}
```



```

.cp-metric-note {font-size:12px; color:#d7e5f8; margin-top:4px;}
.cp-section {font-size: 20px; font-weight:800; color:#fff; margin: 10px 0 10px;}
.cp-note {color:#d8e3f4; font-size:13px;}
.stButton > button {
    background: linear-gradient(135deg, #67e8f9, #60a5fa);
    color: #08111d;
    border: 0;
    border-radius: 12px;
    font-weight: 800;
    box-shadow: 0 10px 24px rgba(96,165,250,0.28);
}
.stButton > button:hover {filter:brightness(1.05);}
[data-testid="stTabs"] [role="tablist"] {
    gap: 8px;
    background: rgba(9,16,30,0.55);
    padding: 8px;
    border-radius: 16px;
    border: 1px solid rgba(255,255,255,0.12);
}
[data-testid="stTabs"] [role="tab"] {
    border-radius: 12px;
    color: #eef6ff;
    font-weight: 800;
    padding: 10px 16px;
    background: rgba(15,23,42,0.62);
    border: 1px solid rgba(255,255,255,0.10);
}
[data-testid="stTabs"] [aria-selected="true"] {
    background: linear-gradient(135deg, rgba(96,165,250,0.30),
    rgba(34,211,238,0.18));
    border: 1px solid rgba(255,255,255,0.24);
}
[data-testid="stPlotlyChart"] {
    border-radius: 18px;
    padding: 10px;
    background: rgba(12,20,37,0.52);
    border: 1px solid rgba(255,255,255,0.14);
    box-shadow: var(--shadow);
}
[data-testid="stTextInput"] > div,
[data-testid="stNumberInput"] > div,
[data-testid="stSelectbox"] > div,
[data-testid="stSlider"] > div,
[data-testid="stTextArea"] > div {
    background: rgba(12,20,37,0.55) !important;
    border-radius: 14px;
}
</style>
"""
unsafe_allow_html=True,
)

```

```

try:
    bg_path = Path(__file__).resolve().parent / "bg.jpeg"
    if bg_path.exists():
        b64 = base64.b64encode(bg_path.read_bytes()).decode()
        st.markdown(
            f"""
            <style>
            .stApp {{

```

```

        background-image: linear-gradient(rgba(3,8,18,.72), rgba(3,8,18,.82)),
url("data:image/jpeg;base64,{b64}");
        background-size: cover;
        background-attachment: fixed;
        background-position: center;
    }}
    [data-testid="stHeader"], [data-testid="stToolbar"] {{
        background: transparent !important;
    }}
</style>
"""
    unsafe_allow_html=True,
)
except Exception:
    pass

def header() -> None:
    st.markdown(
        """
        <div class="cp-hero">
            <div class="cp-title">CreditPilot - Advanced AI Loan Intelligence Suite</div>
            <div class="cp-subtitle">Advisory engine, personalized pricing, prepayment lab,
admin dashboards, history tracking, and a smart credit assistant.</div>
            <div class="cp-pill">Recommendation Engine</div>
            <div class="cp-pill">Interest Estimator</div>
            <div class="cp-pill">Risk Intelligence</div>
            <div class="cp-pill">Portfolio Dashboard</div>
        </div>
        """,
        unsafe_allow_html=True,
    )

def kpi_row(items):
    cols = st.columns(len(items))
    for idx, (label, value, note) in enumerate(items):
        with cols[idx]:
            st.markdown(
                f"""
                <div class="cp-metric">
                    <div class="cp-metric-label">{label}</div>
                    <div class="cp-metric-value">{value}</div>
                    <div class="cp-metric-note">{note}</div>
                </div>
                """,
                unsafe_allow_html=True,
            )

def card(title: str, body: str) -> None:
    st.markdown(
        f"""
        <div class="cp-card">
            <div class="cp-section" style="margin-top:0;">{title}</div>
            <div class="cp-note">{body}</div>
        </div>
        """,
        unsafe_allow_html=True,
    )

```

```

def themed(fig: go.Figure) -> go.Figure:
    fig.update_layout(
        template="plotly_dark",
        paper_bgcolor="rgba(0,0,0,0)",
        plot_bgcolor="rgba(0,0,0,0)",
        font=dict(color="#e5efff"),
        margin=dict(l=20, r=20, t=50, b=20),
    )
    return fig

@st.cache_data(show_spinner=False)
def load_raw_data() -> Optional[pd.DataFrame]:
    if RAW_DATA.exists():
        return pd.read_csv(RAW_DATA)
    return None

@st.cache_data(show_spinner=False)
def load_processed_data() -> Optional[pd.DataFrame]:
    if PROCESSED_DATA.exists():
        return pd.read_csv(PROCESSED_DATA)
    return None

@st.cache_resource(show_spinner=False)
def load_advanced_bundle() -> Optional[Dict[str, Any]]:
    if ADVANCED_BUNDLE.exists() and joblib_load is not None:
        return joblib_load(ADVANCED_BUNDLE)
    return None

def set_sample_state(row: pd.Series) -> None:
    st.session_state["app_name"] = f"Sample User {int(row.get('user_id', 0))}"
    mapping = {
        "main_age": int(row["age"]),
        "main_employment_type": row["employment_type"],
        "main_employment_length": int(row["employment_length_years"]),
        "main_annual_income": float(row["annual_income"]),
        "main_income_stability": float(row["income_stability_score"]),
        "main_loan_amount": float(row["loan_amount"]),
        "main_loan_term": int(row["loan_term_months"]),
        "main_interest_rate": float(row["interest_rate"]),
        "main_loan_type": row["loan_type"],
        "main_existing_loans": int(row["total_existing_loans"]),
        "main_total_debt": float(row["total_debt"]),
        "main_credit_score": int(round(row["credit_score"])),
        "main_utilization": float(row["credit_utilization_ratio"]),
        "main_inquiries": int(row["num_credit_inquiries"]),
        "main_late_payments": int(row["late_payment_count"]),
        "main_default_history": int(row["default_history"]),
        "main_repayment_score": float(row["repayment_history_score"]),
        "main_credit_history": int(row["credit_history_length_years"]),
        "main_job_stability": float(row["job_stability_score"]),
        "main_spending_ratio": float(row["spending_to_income_ratio"]),
    }
    for key, value in mapping.items():
        st.session_state[key] = value

```

```

def applicant_form(raw_df: Optional[pd.DataFrame]) -> Dict[str, Any]:
    st.markdown('<div class="cp-section">Applicant Studio</div>', unsafe_allow_html=True)

    sample_col, button_col, _ = st.columns([1.2, 0.6, 2.2])
    with sample_col:
        sample_index = st.number_input(
            "Load dataset sample",
            min_value=1,
            max_value=int(raw_df.shape[0]) if isinstance(raw_df, pd.DataFrame) else 1,
            value=1,
            step=1,
        )
    with button_col:
        st.write("")
        st.write("")
        if raw_df is not None and st.button("Use Sample"):
            set_sample_state(raw_df.iloc[int(sample_index) - 1])
            st.rerun()

    applicant_name = st.text_input("Applicant Name", value=st.session_state.get("app_name",
"Ujjwal Applicant"), key="app_name")
    c1, c2, c3 = st.columns(3)
    with c1:
        age = st.number_input("Age", 18, 80, value=st.session_state.get("main_age", 32),
key="main_age")
        employment_type = st.selectbox(
            "Employment Type",
            ["salaried", "self-employed", "unemployed"],
            index=["salaried", "self-employed",
"unemployed"].index(st.session_state.get("main_employment_type", "salaried")),
            key="main_employment_type",
        )
        employment_length_years = st.number_input(
            "Employment Length (years)", 0, 40,
value=st.session_state.get("main_employment_length", 6), key="main_employment_length"
        )
        annual_income = st.number_input(
            "Annual Income", 50000, 10000000,
value=int(st.session_state.get("main_annual_income", 900000)), step=10000,
key="main_annual_income"
        )
        income_stability_score = st.slider(
            "Income Stability Score", 0.0, 100.0,
value=float(st.session_state.get("main_income_stability", 72.0)), step=0.5,
key="main_income_stability"
        )
        loan_amount = st.number_input(
            "Loan Amount", 50000, 10000000,
value=int(st.session_state.get("main_loan_amount", 700000)), step=10000,
key="main_loan_amount"
        )
        loan_type = st.selectbox(
            "Loan Type",
            ["personal", "auto", "education", "home"],
            index=["personal", "auto", "education",
"home"].index(st.session_state.get("main_loan_type", "personal")),
            key="main_loan_type",
        )
    with c2:
        loan_term_months = st.selectbox(
            "Loan Term (months)",

```

```

        [12, 24, 36, 48, 60, 84, 120, 180, 240, 300],
        index=[12, 24, 36, 48, 60, 84, 120, 180, 240,
300].index(int(st.session_state.get("main_loan_term", 48))),
        key="main_loan_term",
    )
    interest_rate = st.slider(
        "Current Interest Rate (%)", 5.0, 20.0,
value=float(st.session_state.get("main_interest_rate", 10.8)), step=0.1,
key="main_interest_rate"
    )
    total_existing_loans = st.number_input(
        "Total Existing Loans", 0, 20,
value=st.session_state.get("main_existing_loans", 1), key="main_existing_loans"
    )
    total_debt = st.number_input(
        "Total Debt", 0, 10000000, value=int(st.session_state.get("main_total_debt",
180000)), step=10000, key="main_total_debt"
    )
    credit_score = st.slider("Credit Score", 300, 850,
value=st.session_state.get("main_credit_score", 720), key="main_credit_score")
    credit_utilization_ratio = st.slider(
        "Credit Utilization Ratio", 0.0, 1.0,
value=float(st.session_state.get("main_utilization", 0.32)), step=0.01,
key="main_utilization"
    )
    num_credit_inquiries = st.number_input("Credit Inquiries", 0, 20,
value=st.session_state.get("main_inquiries", 2), key="main_inquiries")
    with c3:
        late_payment_count = st.number_input("Late Payments", 0, 30,
value=st.session_state.get("main_late_payments", 0), key="main_late_payments")
        default_history = st.selectbox("Past Default", [0, 1], index=[0,
1].index(st.session_state.get("main_default_history", 0)), key="main_default_history")
        repayment_history_score = st.slider(
            "Repayment History Score", 0.0, 100.0,
value=float(st.session_state.get("main_repayment_score", 78.0)), step=0.5,
key="main_repayment_score"
        )
        credit_history_length_years = st.number_input(
            "Credit History Length (years)", 0, 40,
value=st.session_state.get("main_credit_history", 7), key="main_credit_history"
        )
        job_stability_score = st.slider(
            "Job Stability Score", 0.0, 100.0,
value=float(st.session_state.get("main_job_stability", 74.0)), step=0.5,
key="main_job_stability"
        )
        spending_to_income_ratio = st.slider(
            "Spending To Income Ratio", 0.0, 1.0,
value=float(st.session_state.get("main_spending_ratio", 0.42)), step=0.01,
key="main_spending_ratio"
        )

    payload = build_payload_from_form(
        age=age,
        employment_type=employment_type,
        employment_length_years=employment_length_years,
        annual_income=annual_income,
        income_stability_score=income_stability_score,
        loan_amount=loan_amount,
        loan_term_months=loan_term_months,
        interest_rate=interest_rate,

```

```

        loan_type=loan_type,
        total_existing_loans=total_existing_loans,
        total_debt=total_debt,
        credit_score=credit_score,
        credit_utilization_ratio=credit_utilization_ratio,
        num_credit_inquiries=num_credit_inquiries,
        late_payment_count=late_payment_count,
        default_history=default_history,
        repayment_history_score=repayment_history_score,
        credit_history_length_years=credit_history_length_years,
        job_stability_score=job_stability_score,
        spending_to_income_ratio=spending_to_income_ratio,
    )
    st.session_state["current_payload"] = payload
    st.session_state["current_applicant_name"] = applicant_name
    return {"name": applicant_name, "payload": payload}

def overview_page(raw_df: Optional[pd.DataFrame], _processed_df: Optional[pd.DataFrame],
bundle: Optional[Dict[str, Any]]) -> None:
    header()
    st.markdown('<div class="cp-section">Overview</div>', unsafe_allow_html=True)
    metrics = bundle.get("metrics", {}) if isinstance(bundle, dict) else {}
    total_records = int(raw_df.shape[0]) if raw_df is not None else 0
    kpi_row(
        [
            ("Records", f"{total_records:,}" if total_records else "-", "Raw dataset
rows"),
            ("Advanced Bundle", "Ready" if bundle is not None else "Missing", "Approval +
rate models"),
            ("Approval Accuracy", f"{metrics.get('approval', {}).get('accuracy', 0):.3f}"
if metrics else "-", "Retrained pipeline"),
            ("Rate MAE", f"{metrics.get('interest', {}).get('mae', 0):.3f}" if metrics else
"-", "Interest estimator error"),
        ]
    )
    c1, c2 = st.columns([1.15, 1])
    with c1:
        card(
            "What Is New",
            "CreditPilot now behaves like an advisory platform. It recommends loan
structures, estimates personalized rates, explains rejections, tests co-applicant impact,
tracks profiles, and exposes admin-grade portfolio analytics.",
        )
    st.markdown('<div class="cp-section">Advanced Feature Map</div>',
unsafe_allow_html=True)
    feature_df = pd.DataFrame(
        {
            "Feature": [
                "Loan recommendation engine",
                "Smart interest estimation",
                "Prepayment and foreclosure analyzer",
                "Co-applicant optimizer",
                "Approval probability graph",
                "Financial goal planner",
                "Risk heatmap and trends",
                "Scenario comparison and history tracker",
            ],
            "Status": ["Live"] * 8,
        }
    )

```

```

        st.dataframe(feature_df, width="stretch", hide_index=True)
    with c2:
        artifact_df = pd.DataFrame(
            {
                "Artifact": ["Raw Data", "Processed Data", "Final Model", "Scaler",
"Advanced Bundle"],
                "Available": [
                    RAW_DATA.exists(),
                    PROCESSED_DATA.exists(),
                    FINAL_MODEL.exists(),
                    SCALER_MODEL.exists(),
                    ADVANCED_BUNDLE.exists(),
                ],
            }
        )
    st.markdown('<div class="cp-section">Artifacts</div>', unsafe_allow_html=True)
    st.dataframe(artifact_df, width="stretch", hide_index=True)
    if raw_df is not None:
        business_snapshot = pd.DataFrame(
            {
                "Metric": [
                    "Average credit score",
                    "Average interest rate",
                    "Average loan amount",
                    "Lowest-risk segment",
                ],
                "Value": [
                    f"{raw_df['credit_score'].mean():.0f}",
                    f"{raw_df['interest_rate'].mean():.2f}%",
                    format_currency(float(raw_df["loan_amount"].mean())),
                ],
            }
        )
        raw_df.groupby("loan_type")["loan_status"].mean().sort_values().index[0].title(),
    ],
    )
    st.markdown('<div class="cp-section">Portfolio Business Snapshot</div>',
unsafe_allow_html=True)
    st.dataframe(business_snapshot, width="stretch", hide_index=True)

def smart_application_page(raw_df: Optional[pd.DataFrame], bundle: Optional[Dict[str,
Any]]) -> None:
    header()
    form = applicant_form(raw_df)
    payload = form["payload"]
    name = form["name"]
    result = evaluate_application(bundle, payload)
    rate_offer, rate_info = estimate_interest_rate(bundle, payload)
    util = credit_utilization_analyzer(payload["credit_utilization_ratio"])

    st.markdown('<div class="cp-section">Decision Engine</div>', unsafe_allow_html=True)
    kpi_row(
        [
            ("Approval Chance", f"{(1 - result['probability']):.1%}",
result["probability_info"].get("mode", "prediction")),
            ("Decision", result["decision"], result["risk"]),
            ("FSI", f"{result['fsi']:.1f}", "Financial stability index"),
            ("EMI", format_currency(result["emi"]), result["affordability"]),
            ("Health Score", f"{result['health_score']:.1f}",
result["early_warning"]["level"] + " warning"),
        ]
    )

```

```

)

c1, c2, c3 = st.columns(3)
with c1:
    card("Personalized Rate Offer", f"Estimated smart rate:
<b>{rate_offer:.2f}%</b><br>Inference: {rate_info.get('mode', 'heuristic')}")
    with c2:
        card("Credit Utilization Analyzer", f"Status:
<b>{util['level']}</b><br>{util['impact']}<br>{util['suggestion']}")
    with c3:
        card("Default Early Warning", f"Level:
<b>{result['early_warning']['level']}</b><br>{result['early_warning']['note']}")

    curve_df = generate_probability_curve(bundle, payload)
    impact_df = variable_impact_table(bundle, payload)
    g1, g2 = st.columns(2)
    with g1:
        fig_curve = px.line(curve_df, x="credit_score", y="approval_chance",
title="Approval Chance vs Credit Score")
        fig_curve.add_vline(x=payload["credit_score"], line_dash="dash",
line_color="#67e8f9")
        themed(fig_curve)
        st.plotly_chart(fig_curve, width="stretch")
    with g2:
        fig_impact = px.bar(impact_df, x="scenario", y="approval_change",
color="approval_change", title="Impact Of Key Variables")
        themed(fig_impact)
        st.plotly_chart(fig_impact, width="stretch")

    rejection_df = pd.DataFrame(result["rejection_breakdown"])
    if not rejection_df.empty:
        top_reason = rejection_df.iloc[0]
        st.markdown('<div class="cp-section">Top Decision Driver</div>',
unsafe_allow_html=True)
        card(top_reason["reason"], f"{top_reason['share']:.1f}%
contribution<br>{top_reason['detail']}")
        with st.expander("View decision driver details"):
            st.dataframe(rejection_df, width="stretch", hide_index=True)

    if raw_df is not None:
        st.markdown('<div class="cp-section">Borrower Benchmark</div>',
unsafe_allow_html=True)
        benchmark_df = applicant_benchmark(payload, raw_df)
        b1, b2 = st.columns([1.05, 1])
        with b1:
            st.dataframe(benchmark_df, width="stretch", hide_index=True)
        with b2:
            fig_bench = px.bar(
                benchmark_df,
                x="metric",
                y="applicant_value",
                color="status",
                title="Applicant vs Portfolio Benchmark",
                hover_data=["portfolio_median"],
            )
            themed(fig_bench)
            st.plotly_chart(fig_bench, width="stretch")

        st.markdown('<div class="cp-section">Financial Health Suggestions</div>',
unsafe_allow_html=True)
        for item in result["suggestions"][:6]:

```



```

        st.markdown(f"- {item}")

save_col, _ = st.columns([0.45, 2.55])
with save_col:
    if st.button("Save Profile Snapshot"):
        save_profile_event(
            {
                "timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
                "name": name,
                "loan_type": payload["loan_type"],
                "loan_amount": payload["loan_amount"],
                "approval_chance": round((1 - result["probability"]) * 100, 2),
                "decision": result["decision"],
                "risk": result["risk"],
            }
        )
        st.success("Profile snapshot saved.")

def recommendation_page(bundle: Optional[Dict[str, Any]]) -> None:
    header()
    payload = st.session_state.get("current_payload")
    if not payload:
        st.info("Open Smart Application first and enter an applicant profile.")
        return

    rec_df = recommend_loans(bundle, payload)
    top = rec_df.iloc[0]
    st.markdown('<div class="cp-section">Loan Recommendation Engine</div>',
unsafe_allow_html=True)
    kpi_row(
        [
            ("Best Loan Type", top["loan_type"], "Highest utility fit"),
            ("Optimal Tenure", f"{int(top['loan_term_months'])} months", "Recommended
structure"),
            ("Optimal Amount", format_currency(float(top["loan_amount"])), "Safer offer
size"),
            ("Expected Approval", f"{float(top['approval_chance']):.1f}%",
top["decision"]),
        ]
    )

    c1, c2 = st.columns([1.25, 1])
    with c1:
        st.dataframe(
            rec_df[["loan_type", "loan_amount", "loan_term_months", "interest_rate", "emi",
"approval_chance", "decision"]],
            width="stretch",
            hide_index=True,
        )
    with c2:
        fig = px.bar(rec_df.head(8), x="loan_type", y="approval_chance",
color="utility_score", title="Top Recommended Structures")
        themed(fig)
        st.plotly_chart(fig, width="stretch")

    st.markdown('<div class="cp-section">Co-Applicant Optimization</div>',
unsafe_allow_html=True)
    a1, a2, a3 = st.columns(3)
    with a1:
        co_income = st.number_input("Co-applicant income", 0, 5000000, 250000, step=10000)

```

```

with a2:
    co_score = st.slider("Co-applicant credit score", 300, 850, 730)
with a3:
    debt_share = st.slider("Debt share reduction", 0.0, 0.6, 0.2, 0.05)
co_result = coapplicant_optimization(bundle, payload, co_income, co_score, debt_share)
kpi_row(
    [
        ("Base Approval", f"{(1 - co_result['base']['probability']):.1%}",
co_result["base"]["decision"]),
        ("With Co-applicant", f"{(1 -
co_result['with_coapplicant']['probability']):.1%}",
co_result["with_coapplicant"]["decision"]),
        ("Approval Gain", f"{co_result['approval_gain']:.1f} pts", "Improvement"),
        ("Recommendation", "Add Co-applicant" if co_result["should_add"] else
"Optional", "Optimization view"),
    ]
)

st.markdown('<div class="cp-section">Multi-Loan Scenario Comparison</div>',
unsafe_allow_html=True)
s1, s2 = st.columns(2)
with s1:
    type_a = st.selectbox("Loan A Type", ["personal", "auto", "education", "home"],
index=0)
    amount_a = st.number_input("Loan A Amount", 50000, 5000000,
int(payload["loan_amount"]), step=10000)
    term_a = st.selectbox("Loan A Tenure", [12, 24, 36, 48, 60, 84, 120, 180, 240,
300], index=3)
    with s2:
        type_b = st.selectbox("Loan B Type", ["personal", "auto", "education", "home"],
index=1)
        amount_b = st.number_input("Loan B Amount", 50000, 5000000,
int(payload["loan_amount"] * 0.9), step=10000)
        term_b = st.selectbox("Loan B Tenure", [12, 24, 36, 48, 60, 84, 120, 180, 240,
300], index=4)

scenario_a = dict(payload)
scenario_a["loan_type"] = type_a
scenario_a["loan_amount"] = float(amount_a)
scenario_a["loan_term_months"] = int(term_a)
scenario_a["interest_rate"], _ = estimate_interest_rate(bundle, scenario_a)

scenario_b = dict(payload)
scenario_b["loan_type"] = type_b
scenario_b["loan_amount"] = float(amount_b)
scenario_b["loan_term_months"] = int(term_b)
scenario_b["interest_rate"], _ = estimate_interest_rate(bundle, scenario_b)

comparison = compare_loan_scenarios(
    bundle,
    [
        {"label": "Loan A", "payload": scenario_a},
        {"label": "Loan B", "payload": scenario_b},
    ],
)
st.dataframe(comparison, width="stretch", hide_index=True)

def emi_page() -> None:
    header()

```

```

    st.markdown('<div class="cp-section">EMI Calculator And Prepayment Planner</div>',
unsafe_allow_html=True)
    c1, c2 = st.columns([1, 1])
    with c1:
        loan_amount = st.number_input("Planner Loan Amount", 50000.0, 10000000.0, 800000.0,
step=10000.0)
        annual_rate = st.slider("Planner Interest Rate (%)", 5.0, 25.0, 10.5, 0.1)
        tenure_years = st.slider("Planner Tenure (years)", 1, 30, 5)
        extra_payment = st.number_input("Extra Monthly Payment", 0.0, 200000.0, 5000.0,
step=500.0)
        months = tenure_years * 12
        emi = calculate_emi(loan_amount, annual_rate, months)
        schedule = amortization_schedule(loan_amount, annual_rate, months,
extra_payment=extra_payment)
        summary = prepayment_summary(loan_amount, annual_rate, months, extra_payment)
    with c2:
        kpi_row(
            [
                ("Live EMI", format_currency(emi), "Updates instantly"),
                ("Interest Saved", format_currency(summary["interest_saved"]), "With
prepayment"),
                ("Tenure Reduced", f"{summary['months_saved']} months", "Foreclosure
benefit"),
                ("Total Interest", format_currency(summary["new_interest"]), "After extra
payment"),
            ]
        )

    g1, g2 = st.columns(2)
    with g1:
        fig_balance = px.line(schedule, x="month", y="balance", title="Remaining Balance
Trend")
        themed(fig_balance)
        st.plotly_chart(fig_balance, width="stretch")
    with g2:
        fig_mix = px.area(schedule, x="month", y=["principal", "interest"],
title="Principal vs Interest Mix")
        themed(fig_mix)
        st.plotly_chart(fig_mix, width="stretch")
    with st.expander("View Full Amortization Schedule"):
        st.dataframe(schedule, width="stretch", hide_index=True)

def fsi_page(bundle: Optional[Dict[str, Any]]) -> None:
    header()
    payload = st.session_state.get("current_payload")
    if not payload:
        st.info("Open Smart Application first and enter an applicant profile.")
        return

    result = evaluate_application(bundle, payload)
    model_payload = result["payload"]
    component_df = pd.DataFrame(
        [
            {"component": "Income Stability", "score":
round(model_payload["income_stability_score"], 2)},
            {"component": "Credit Strength", "score": round(((model_payload["credit_score"]
- 300) / (850 - 300)) * 100, 2)},
            {"component": "Debt Headroom", "score": round(max(0.0, 1 -
model_payload["debt_to_income_ratio"]) * 100, 2)},

```

```

        {"component": "EMI Headroom", "score": round(max(0.0, 1 -
model_payload["emi_to_income_ratio"]) * 100, 2)},
        {"component": "Utilization Headroom", "score": round(max(0.0, 1 -
model_payload["credit_utilization_ratio"]) * 100, 2)},
    ]
)

st.markdown('<div class="cp-section">Financial Stability Index</div>',
unsafe_allow_html=True)
kpi_row(
    [
        ("FSI", f"{result['fsi']:.1f}", "Composite resilience score"),
        ("Risk Class", result["risk"], "Current borrower posture"),
        ("Affordability", result["affordability"], "Monthly capacity view"),
        ("Health Score", f"{result['health_score']:.1f}", "Combined profile health"),
    ]
)

g1, g2 = st.columns(2)
with g1:
    fig_gauge = go.Figure(
        go.Indicator(
            mode="gauge+number",
            value=result["fsi"],
            title={"text": "FSI Gauge"},
            number={"suffix": "/100"},
            gauge={
                "axis": {"range": [0, 100]},
                "bar": {"color": "#67e8f9"},
                "steps": [
                    {"range": [0, 50], "color": "rgba(239,68,68,0.35)"},
                    {"range": [50, 70], "color": "rgba(250,204,21,0.30)"},
                    {"range": [70, 100], "color": "rgba(74,222,128,0.30)"},
                ],
            },
        )
    )
    themed(fig_gauge)
    st.plotly_chart(fig_gauge, width="stretch")
with g2:
    fig_components = px.bar(component_df, x="component", y="score", color="score",
title="FSI Component Breakdown")
    fig_components.update_layout(xaxis_title="", yaxis_title="Score")
    themed(fig_components)
    st.plotly_chart(fig_components, width="stretch")

fig_radar = go.Figure()
fig_radar.add_trace(
    go.Scatterpolar(
        r=component_df["score"].tolist(),
        theta=component_df["component"].tolist(),
        fill="toself",
        name="FSI mix",
        line=dict(color="#60a5fa"),
    )
)
fig_radar.update_layout(title="Financial Stability Radar",
polar=dict(radialaxis=dict(range=[0, 100])))
themed(fig_radar)
st.plotly_chart(fig_radar, width="stretch")

```

```

st.markdown('<div class="cp-section">Stability Notes</div>', unsafe_allow_html=True)
notes_df = pd.DataFrame(
    [
        {"metric": "Credit score", "value": model_payload["credit_score"], "note":
"Higher values improve FSI."},
        {"metric": "Debt-to-income", "value":
round(model_payload["debt_to_income_ratio"], 2), "note": "Lower leverage improves
resilience."},
        {"metric": "EMI-to-income", "value":
round(model_payload["emi_to_income_ratio"], 2), "note": "High monthly pressure reduces
stability."},
        {"metric": "Credit utilization", "value":
round(model_payload["credit_utilization_ratio"], 2), "note": "Lower revolving usage is
healthier."},
    ]
)
st.dataframe(notes_df, width="stretch", hide_index=True)

def stress_lab_page(bundle: Optional[Dict[str, Any]]) -> None:
    header()
    payload = st.session_state.get("current_payload")
    if not payload:
        st.info("Open Smart Application first and enter an applicant profile.")
        return

    base_result = evaluate_application(bundle, payload)
    st.markdown('<div class="cp-section">Stress Lab</div>', unsafe_allow_html=True)

    c1, c2, c3, c4 = st.columns(4)
    with c1:
        income_shock = st.slider("Income Shock (%)", -50, 20, -10, 5)
    with c2:
        rate_shock = st.slider("Rate Shock (%)", 0.0, 6.0, 1.0, 0.1)
    with c3:
        debt_shock = st.slider("Debt Shock (%)", 0, 100, 20, 5)
    with c4:
        util_shock = st.slider("Utilization Shock", 0.0, 0.5, 0.1, 0.01)

    stressed_payload = dict(payload)
    stressed_payload["annual_income"] = max(50000.0, payload["annual_income"] * (1 +
income_shock / 100))
    stressed_payload["interest_rate"] = min(25.0, payload["interest_rate"] + rate_shock)
    stressed_payload["total_debt"] = max(0.0, payload["total_debt"] * (1 + debt_shock /
100))
    stressed_payload["credit_utilization_ratio"] = min(1.0,
payload["credit_utilization_ratio"] + util_shock)
    stressed_result = evaluate_application(bundle, stressed_payload)
    summary = stress_delta_summary(base_result["probability"],
stressed_result["probability"])

    kpi_row(
        [
            ("Base Approval", f"{summary['base_approval']:.1f}%", base_result["decision"]),
            ("Stress Approval", f"{summary['stress_approval']:.1f}%",
stressed_result["decision"]),
            ("Stress Impact", summary["impact"], "Scenario effect"),
            ("FSI Delta", f"{stressed_result['fsi'] - base_result['fsi']:.1f}%",
"Resilience change"),
            ("EMI Delta", format_currency(stressed_result["emi"] - base_result["emi"]),
"Monthly payment shift"),

```

```

    ]
)

compare_df = pd.DataFrame(
    [
        {"metric": "Approval chance", "base": summary["base_approval"], "stress":
summary["stress_approval"]},
        {"metric": "FSI", "base": base_result["fsi"], "stress":
stressed_result["fsi"]},
        {"metric": "EMI", "base": base_result["emi"], "stress":
stressed_result["emi"]},
        {"metric": "Health score", "base": base_result["health_score"], "stress":
stressed_result["health_score"]},
    ]
)

compare_long = compare_df.melt(id_vars="metric", var_name="scenario",
value_name="value")

g1, g2 = st.columns(2)
with g1:
    fig_compare = px.bar(compare_long, x="metric", y="value", color="scenario",
barmode="group", title="Base vs Stress Comparison")
    themed(fig_compare)
    st.plotly_chart(fig_compare, width="stretch")
with g2:
    shock_rows = []
    for shock in range(0, 41, 5):
        sim = dict(payload)
        sim["annual_income"] = max(50000.0, payload["annual_income"] * (1 - shock /
100))

        sim_result = evaluate_application(bundle, sim)
        shock_rows.append(
            {
                "income_drop_pct": shock,
                "approval_chance": round((1 - sim_result["probability"]) * 100, 2),
                "fsi": sim_result["fsi"],
            }
        )

    shock_df = pd.DataFrame(shock_rows)
    fig_curve = px.line(shock_df, x="income_drop_pct", y=["approval_chance", "fsi"],
markers=True, title="Sensitivity To Income Stress")
    themed(fig_curve)
    st.plotly_chart(fig_curve, width="stretch")

details_df = pd.DataFrame(
    [
        {"field": "Annual income", "base": format_currency(payload["annual_income"]),
"stress": format_currency(stressed_payload["annual_income"])},
        {"field": "Interest rate", "base": f"{payload['interest_rate']:.2f}%",
"stress": f"{stressed_payload['interest_rate']:.2f}%",
        {"field": "Total debt", "base": format_currency(payload["total_debt"]),
"stress": format_currency(stressed_payload["total_debt"])},
        {"field": "Credit utilization", "base":
f"{payload['credit_utilization_ratio']:.2f}", "stress":
f"{stressed_payload['credit_utilization_ratio']:.2f}"},
    ]
)

st.dataframe(details_df, width="stretch", hide_index=True)

def goal_planner_page(bundle: Optional[Dict[str, Any]]) -> None:

```

```

header()
payload = st.session_state.get("current_payload")
if not payload:
    st.info("Open Smart Application first and enter an applicant profile.")
    return
st.markdown('<div class="cp-section">Financial Goal Planner</div>',
unsafe_allow_html=True)
c1, c2, c3 = st.columns(3)
with c1:
    goal_type = st.selectbox("Goal Type", ["Buy House", "Buy Car", "Higher Education",
"Business Expansion"])
    with c2:
        years_to_goal = st.slider("Years To Goal", 1, 15, 4)
    with c3:
        down_payment = st.number_input("Planned Down Payment", 0, 5000000, 250000,
step=10000)
    plan = financial_goal_plan(bundle, payload, goal_type, down_payment, years_to_goal)

    loan_type_map = {
        "Buy House": "home",
        "Buy Car": "auto",
        "Higher Education": "education",
        "Business Expansion": "personal",
    }
    goal_payload = dict(payload)
    goal_payload["loan_type"] = loan_type_map[goal_type]
    goal_payload["loan_amount"] = float(plan["safe_loan_amount"])
    goal_payload["loan_term_months"] = 240 if goal_type == "Buy House" else (60 if
goal_type == "Buy Car" else 84)
    goal_payload["interest_rate"], _ = estimate_interest_rate(bundle, goal_payload)
    goal_eval = evaluate_application(bundle, goal_payload)

    kpi_row(
        [
            ("Safe Loan Amount", format_currency(plan["safe_loan_amount"]), goal_type),
            ("Comfortable EMI", format_currency(plan["comfortable_emi"]), "Safe monthly
zone"),
            ("Projected Approval", f"{{(1 - goal_eval['probability']):.1%}}",
goal_eval["decision"]),
            ("Recommended Rate", f"{{goal_payload['interest_rate']:.2f}}%", "Estimated
offer"),
        ]
    )
    card("Planner Insight", plan["note"])

def admin_dashboard_page(raw_df: Optional[pd.DataFrame]) -> None:
    header()
    if raw_df is None:
        st.error("Raw dataset not found.")
        return
    st.markdown('<div class="cp-section">Admin Dashboards</div>', unsafe_allow_html=True)
    portfolio = portfolio_summary(raw_df)
    kpi_row(
        [
            ("Portfolio Size", f"{{portfolio['portfolio_size']:,}}", "All applications"),
            ("Approved Loans", f"{{portfolio['total_approved']:,}}", "Estimated from
dataset"),
            ("Approval Rate", f"{{portfolio['approval_rate']:.1%}}", "Portfolio conversion"),
            ("High Risk Share", f"{{portfolio['high_risk_share']:.1%}}",
portfolio["warning"]),

```

```

    ]
)

heatmap_df = risk_heatmap_frame(raw_df)
trend_df = approval_trend_frame(raw_df)

raw_work = raw_df.copy()
raw_work["risk_cluster"] = raw_work["credit_score"].apply(lambda x: "High Risk" if x <
650 else ("Medium Risk" if x < 720 else "Low Risk"))

g1, g2 = st.columns(2)
with g1:
    fig_heat = px.imshow(
        heatmap_df,
        text_auto=".0%",
        color_continuous_scale="RdYlBu_r",
        title="Risk Heatmap: Default Rate by DTI and Credit Band",
    )
    themed(fig_heat)
    st.plotly_chart(fig_heat, width="stretch")
with g2:
    fig_cluster = px.scatter(
        raw_work.sample(min(1200, len(raw_work)), random_state=42),
        x="credit_score",
        y="debt_to_income_ratio",
        color="risk_cluster",
        size="loan_amount",
        title="High-Risk Cluster View",
        hover_data=["loan_type", "interest_rate"],
    )
    themed(fig_cluster)
    st.plotly_chart(fig_cluster, width="stretch")

fig_trend = px.bar(trend_df, x="loan_type", y="approval_rate",
color="avg_interest_rate", title="Approval Trend by Loan Type")
themed(fig_trend)
st.plotly_chart(fig_trend, width="stretch")

def profile_tracker_page() -> None:
    header()
    st.markdown('<div class="cp-section">User Profile And Loan History Tracker</div>',
unsafe_allow_html=True)
    history = load_profile_history()
    history_df = build_history_frame()
    if not history:
        st.info("No saved profile snapshots yet. Save one from Smart Application.")
        return
    kpi_row(
        [
            ("Saved Profiles", f"{len(history):,}", "Snapshots stored locally"),
            ("Latest Applicant", history[-1].get("name", "-"), "Most recent"),
            ("Latest Decision", history[-1].get("decision", "-"), history[-1].get("risk",
"-")),
            ("Latest Approval", f"{history[-1].get('approval_chance', 0):.1f}%", "Chance at
save time"),
        ]
    )
    st.dataframe(history_df, width="stretch", hide_index=True)
    if len(history_df) > 1:

```



```

fig_hist = px.line(history_df, x="timestamp", y="approval_chance", color="name",
markers=True, title="Approval Trend Over Time")
themed(fig_hist)
st.plotly_chart(fig_hist, width="stretch")

def assistant_page(bundle: Optional[Dict[str, Any]]) -> None:
    header()
    payload = st.session_state.get("current_payload")
    st.markdown('<div class="cp-section">Smart Credit Assistant</div>',
unsafe_allow_html=True)
    c1, c2 = st.columns([1.05, 1.45])
    with c1:
        choice = st.radio(
            "Choose a guidance action",
            [
                "Loan eligibility help",
                "Improve approval chances",
                "Best loan type",
                "Check EMI",
                "Can I lower my EMI",
                "Should I add co-applicant",
                "Rate negotiation tips",
            ],
        )
    tone = st.selectbox("Assistant mode", ["Executive summary", "Detailed advisor"],
index=0)
    if payload is None:
        st.info("Load a borrower profile in Smart Application to unlock personalized
assistant guidance.")
    response = assistant_response(choice, payload, bundle)
    with c2:
        card(response["title"], response["answer"])
        if response["highlights"]:
            kpi_row([(label, value, "Assistant insight") for label, value in
response["highlights"]])
        st.markdown('<div class="cp-section">Action Points</div>', unsafe_allow_html=True)
        if response["bullets"]:
            bullets = response["bullets"] if tone == "Detailed advisor" else
response["bullets"][:3]
            for item in bullets:
                st.markdown(f"- {item}")
        else:
            st.markdown("- Add a borrower profile to get specific recommendations.")

def visualisations_page(raw_df: Optional[pd.DataFrame], bundle: Optional[Dict[str, Any]]) -
> None:
    header()
    if raw_df is None:
        st.error("Raw dataset not found.")
        return

    st.markdown('<div class="cp-section">Visualisations</div>', unsafe_allow_html=True)
    work = raw_df.copy()
    work[work["loan_type"] != "home"]
    work["approval_flag"] = 1 - work["loan_status"]
    work["decision"] = work["approval_flag"].map({1: "Approved", 0: "Rejected"})
    work["risk_cluster"] = work["credit_score"].apply(lambda x: "High Risk" if x < 650 else
("Medium Risk" if x < 720 else "Low Risk"))

```

```

g1, g2 = st.columns(2)
with g1:
    approval_by_type = work.groupby("loan_type",
as_index=False)["approval_flag"].mean()
    fig_approval = px.bar(approval_by_type, x="loan_type", y="approval_flag",
color="approval_flag", title="Approval Rate By Loan Type")
    fig_approval.update_layout(yaxis_tickformat=".0%")
    themed(fig_approval)
    st.plotly_chart(fig_approval, width="stretch")
with g2:
    fig_score = px.histogram(
        work,
        x="credit_score",
        color="decision",
        nbins=30,
        barmode="overlay",
        title="Credit Score Distribution By Decision",
    )
    themed(fig_score)
    st.plotly_chart(fig_score, width="stretch")

g3, g4 = st.columns(2)
with g3:
    scatter_df = work.sample(min(1200, len(work)), random_state=42)
    fig_scatter = px.scatter(
        scatter_df,
        x="credit_score",
        y="interest_rate",
        color="loan_type",
        size="loan_amount",
        title="Credit Score vs Interest Rate",
        hover_data=["annual_income", "decision"],
    )
    themed(fig_scatter)
    st.plotly_chart(fig_scatter, width="stretch")
with g4:
    fig_income = px.scatter(
        scatter_df,
        x="annual_income",
        y="loan_amount",
        color="risk_cluster",
        title="Income vs Loan Amount",
        hover_data=["loan_type", "credit_score"],
    )
    themed(fig_income)
    st.plotly_chart(fig_income, width="stretch")

g5, g6 = st.columns(2)
with g5:
    fig_box = px.box(work, x="employment_type", y="debt_to_income_ratio",
color="employment_type", title="Debt-To-Income By Employment Type")
    themed(fig_box)
    st.plotly_chart(fig_box, width="stretch")
with g6:
    heatmap_source = work.pivot_table(
        index="employment_type",
        columns="loan_type",
        values="approval_flag",
        aggfunc="mean",
        observed=False,
    ).fillna(0.0)

```

```

fig_heat = px.imshow(
    heatmap_source,
    text_auto=".0%",
    color_continuous_scale="Blues",
    title="Approval Heatmap: Employment vs Loan Type",
)
themed(fig_heat)
st.plotly_chart(fig_heat, width="stretch")

g7, g8 = st.columns(2)
with g7:
    fig_pie = px.pie(work, names="employment_type", hole=0.55, title="Employment Mix")
    themed(fig_pie)
    st.plotly_chart(fig_pie, width="stretch")
with g8:
    corr_cols = [
        "annual_income",
        "loan_amount",
        "interest_rate",
        "credit_score",
        "debt_to_income_ratio",
        "credit_utilization_ratio",
        "repayment_history_score",
    ]
    corr_df = work[corr_cols].corr().round(2)
    fig_corr = px.imshow(corr_df, text_auto=True, color_continuous_scale="RdBu_r",
title="Correlation Matrix")
    themed(fig_corr)
    st.plotly_chart(fig_corr, width="stretch")

payload = st.session_state.get("current_payload")
if payload:
    result = evaluate_application(bundle, payload)
    rejection_df = pd.DataFrame(result["rejection_breakdown"])
    applicant_curve = generate_probability_curve(bundle, payload)
    v1, v2 = st.columns(2)
    with v1:
        fig_curve = px.line(applicant_curve, x="credit_score", y="approval_chance",
title="Applicant Approval Curve")
        fig_curve.add_vline(x=payload["credit_score"], line_dash="dash",
line_color="#67e8f9")
        themed(fig_curve)
        st.plotly_chart(fig_curve, width="stretch")
    with v2:
        fig_rej = px.bar(rejection_df, x="reason", y="share", text="share",
color="share", title="Applicant Decision Driver Breakdown")
        fig_rej.update_traces(texttemplate="%{text:.1f}%")
        themed(fig_rej)
        st.plotly_chart(fig_rej, width="stretch")

def main() -> None:
    _inject_style()
    raw_df = load_raw_data()
    processed_df = load_processed_data()
    bundle = load_advanced_bundle()

    tabs = st.tabs(
        [
            "Overview",
            "Smart Application",

```

```

        "Recommendation Lab",
        "Stress Lab",
        "FSI",
        "EMI Calculator",
        "Visualisations",
        "Goal Planner",
        "Admin Dashboards",
        "Profile Tracker",
        "Assistant",
    ]
)

with tabs[0]:
    overview_page(raw_df, processed_df, bundle)
with tabs[1]:
    smart_application_page(raw_df, bundle)
with tabs[2]:
    recommendation_page(bundle)
with tabs[3]:
    stress_lab_page(bundle)
with tabs[4]:
    fsi_page(bundle)
with tabs[5]:
    emi_page()
with tabs[6]:
    visualisations_page(raw_df, bundle)
with tabs[7]:
    goal_planner_page(bundle)
with tabs[8]:
    admin_dashboard_page(raw_df)
with tabs[9]:
    profile_tracker_page()
with tabs[10]:
    assistant_page(bundle)

if __name__ == "__main__":
    main()

```

2. Src/logic.py

```

import numpy as np

# -----
# 1. EMI CALCULATOR
# -----
def calculate_emi(loan_amount, interest_rate, tenure):
    monthly_rate = interest_rate / (12 * 100)

    emi = (loan_amount * monthly_rate * (1 + monthly_rate) ** tenure) / \
        ((1 + monthly_rate) ** tenure - 1)

    return round(emi, 2)

# -----
# 2. FEATURE ENGINEERING
# -----

```

```

def prepare_features(data):
    annual_income = data["annual_income"]
    loan_amount = data["loan_amount"]
    emi = data["emi"]
    total_debt = data["total_debt"]

    data["emi_to_income_ratio"] = emi / (annual_income / 12)
    data["loan_to_income_ratio"] = loan_amount / annual_income
    data["debt_to_income_ratio"] = total_debt / annual_income

    data["credit_risk_score"] = (
        (850 - data["credit_score"]) * 0.4 +
        data["debt_to_income_ratio"] * 100 * 0.3 +
        data["credit_utilization_ratio"] * 100 * 0.3
    )

    return data

# -----
# 3. FINANCIAL STABILITY INDEX
# -----
def calculate_fsi(data):
    credit_norm = (data["credit_score"] - 300) / (850 - 300)

    dti_score = 1 - data["debt_to_income_ratio"]
    emi_score = 1 - data["emi_to_income_ratio"]
    util_score = 1 - data["credit_utilization_ratio"]

    fsi = (
        0.25 * (data["income_stability_score"] / 100) +
        0.20 * credit_norm +
        0.20 * dti_score +
        0.15 * emi_score +
        0.20 * util_score
    ) * 100

    return round(fsi, 2)

# -----
# 4. RISK CATEGORY
# -----
def get_risk(prob, fsi):
    if prob > 0.7 or fsi < 50:
        return "High Risk"
    elif prob > 0.4 or fsi < 65:
        return "Medium Risk"
    else:
        return "Low Risk"

# -----
# 5. FINAL DECISION
# -----
def final_decision(prob, fsi):
    if prob < 0.4 and fsi > 70:
        return "APPROVED"
    elif prob < 0.6 and fsi > 55:
        return "CONDITIONAL"
    else:

```

```

        return "REJECTED"

# -----
# 6. AFFORDABILITY CHECK
# -----
def check_affordability(emi, monthly_income):
    ratio = emi / monthly_income

    if ratio < 0.3:
        return "Affordable"
    elif ratio < 0.5:
        return "Moderate Risk"
    else:
        return "Not Affordable"

# -----
# 7. EXPLAIN DECISION (XAI)
# -----
def explain_decision(data, prob, fsi):
    reasons = []

    if data["credit_score"] < 650:
        reasons.append("Low credit score")

    if data["debt_to_income_ratio"] > 0.4:
        reasons.append("High debt-to-income ratio")

    if data["emi_to_income_ratio"] > 0.4:
        reasons.append("High EMI burden")

    if data["credit_utilization_ratio"] > 0.6:
        reasons.append("High credit utilization")

    if prob > 0.6:
        reasons.append("High default probability")

    if fsi < 60:
        reasons.append("Low financial stability")

    return reasons

# -----
# 8. IMPROVEMENT SUGGESTIONS
# -----
def suggest_improvements(data):
    suggestions = []

    if data["credit_score"] < 700:
        suggestions.append("Improve credit score with timely payments")

    if data["debt_to_income_ratio"] > 0.4:
        suggestions.append("Reduce existing debt obligations")

    if data["emi_to_income_ratio"] > 0.4:
        suggestions.append("Consider reducing loan amount")

    if data["credit_utilization_ratio"] > 0.6:
        suggestions.append("Lower credit utilization")

```

```

    if data["income_stability_score"] < 60:
        suggestions.append("Increase income stability")

    return suggestions

# -----
# 9. LOAN HEALTH SCORE
# -----
def loan_health_score(prob, fsi):
    score = (1 - prob) * 50 + (fsi / 100) * 50
    return round(score, 2)

# -----
# 10. STRESS IMPACT ANALYSIS
# -----
def stress_impact(base_prob, stress_prob):
    change = stress_prob - base_prob

    if change > 0.3:
        return "Severely Impacted"
    elif change > 0.1:
        return "Moderately Impacted"
    else:
        return "Stable"

```

3. Src/advanced_logic.py

```

import json
from pathlib import Path
from typing import Any, Dict, List, Optional, Tuple

import numpy as np
import pandas as pd

```

```

from logic import (
    calculate_emi,
    calculate_fsi,
    check_affordability,
    final_decision,
    get_risk,
    loan_health_score,
    prepare_features,
    stress_impact,
    suggest_improvements,
)

```

```

BASE_DIR = Path(__file__).resolve().parent.parent
HISTORY_DIR = BASE_DIR / "data" / "app_history"
HISTORY_FILE = HISTORY_DIR / "profile_history.json"

```

```

LOAN_CONFIG: Dict[str, Dict[str, Any]] = {
    "personal": {"label": "Personal", "terms": [12, 24, 36, 48, 60], "cap_multiple": 0.45},
    "auto": {"label": "Car", "terms": [24, 36, 48, 60, 84], "cap_multiple": 0.6},
    "education": {"label": "Education", "terms": [24, 36, 48, 60, 84, 120], "cap_multiple":
0.5},

```

```

    "home": {"label": "Home", "terms": [120, 180, 240, 300], "cap_multiple": 0.9},
}

def format_currency(value: float) -> str:
    return f"Rs. {value:,.0f}"

def to_model_payload(payload: Dict[str, Any], loan_type: Optional[str] = None) -> Dict[str, Any]:
    data = dict(payload)
    if loan_type is not None:
        data["loan_type"] = loan_type
    data["emi"] = calculate_emi(data["loan_amount"], data["interest_rate"],
int(data["loan_term_months"]))
    data = prepare_features(data)
    return data

def build_payload_from_form(
    age: int,
    employment_type: str,
    employment_length_years: int,
    annual_income: float,
    income_stability_score: float,
    loan_amount: float,
    loan_term_months: int,
    interest_rate: float,
    loan_type: str,
    total_existing_loans: int,
    total_debt: float,
    credit_score: int,
    credit_utilization_ratio: float,
    num_credit_inquiries: int,
    late_payment_count: int,
    default_history: int,
    repayment_history_score: float,
    credit_history_length_years: int,
    job_stability_score: float,
    spending_to_income_ratio: float,
) -> Dict[str, Any]:
    return {
        "age": int(age),
        "employment_type": employment_type,
        "employment_length_years": int(employment_length_years),
        "annual_income": float(annual_income),
        "income_stability_score": float(income_stability_score),
        "loan_amount": float(loan_amount),
        "loan_term_months": int(loan_term_months),
        "interest_rate": float(interest_rate),
        "loan_type": loan_type,
        "total_existing_loans": int(total_existing_loans),
        "total_debt": float(total_debt),
        "credit_score": int(credit_score),
        "credit_utilization_ratio": float(credit_utilization_ratio),
        "num_credit_inquiries": int(num_credit_inquiries),
        "late_payment_count": int(late_payment_count),
        "default_history": int(default_history),
        "repayment_history_score": float(repayment_history_score),
        "credit_history_length_years": int(credit_history_length_years),
        "job_stability_score": float(job_stability_score),
    }

```



```

    "spending_to_income_ratio": float(spending_to_income_ratio),
}

def build_feature_frame(payload: Dict[str, Any]) -> pd.DataFrame:
    data = to_model_payload(payload)
    cols = [
        "age",
        "employment_type",
        "employment_length_years",
        "annual_income",
        "income_stability_score",
        "loan_amount",
        "loan_term_months",
        "interest_rate",
        "loan_type",
        "emi",
        "total_existing_loans",
        "total_debt",
        "debt_to_income_ratio",
        "credit_score",
        "credit_utilization_ratio",
        "emi_to_income_ratio",
        "loan_to_income_ratio",
        "num_credit_inquiries",
        "late_payment_count",
        "default_history",
        "repayment_history_score",
        "credit_history_length_years",
        "job_stability_score",
        "spending_to_income_ratio",
        "credit_risk_score",
    ]
    return pd.DataFrame([col: data[col] for col in cols])

def _bundle_metrics(bundle: Optional[Dict[str, Any]]) -> Dict[str, Any]:
    if isinstance(bundle, dict):
        return bundle.get("metrics", {})
    return {}

def predict_approval_probability(bundle: Optional[Dict[str, Any]], payload: Dict[str, Any])
-> Tuple[float, Dict[str, Any]]:
    info: Dict[str, Any] = {"mode": "logic_fallback"}
    model_payload = to_model_payload(payload)
    if isinstance(bundle, dict) and bundle.get("approval_model") is not None:
        features = pd.DataFrame([model_payload])
        try:
            prob = float(bundle["approval_model"].predict_proba(features)[0, 1])
            info["mode"] = "advanced_bundle"
            info["model_metrics"] = _bundle_metrics(bundle).get("approval")
            return prob, info
        except Exception as exc:
            info["bundle_error"] = str(exc)
    risk = (model_payload["credit_risk_score"] - 50) / 40.0
    return float(1 / (1 + np.exp(-risk))), info

def estimate_interestRate(bundle: Optional[Dict[str, Any]], payload: Dict[str, Any]) ->
Tuple[float, Dict[str, Any]]:

```

```

info: Dict[str, Any] = {"mode": "heuristic_rate"}
model_payload = to_model_payload(payload)
if isinstance(bundle, dict) and bundle.get("interest_model") is not None:
    features = pd.DataFrame([model_payload])
    try:
        rate = float(bundle["interest_model"].predict(features)[0])
        rate = float(np.clip(rate, 6.5, 18.5))
        info["mode"] = "advanced_bundle"
        info["model_metrics"] = _bundle_metrics(bundle).get("interest")
        return rate, info
    except Exception as exc:
        info["bundle_error"] = str(exc)

base_rate = 8.0
score_penalty = max(0.0, (700 - model_payload["credit_score"]) / 55.0)
dti_penalty = max(0.0, (model_payload["debt_to_income_ratio"] - 0.35) * 7.5)
util_penalty = max(0.0, (model_payload["credit_utilization_ratio"] - 0.3) * 6.0)
stability_discount = max(0.0, (model_payload["income_stability_score"] - 60) / 40.0)
loan_type_penalty = {"personal": 2.2, "auto": 1.6, "education": 1.3, "home": 0.8}
rate = base_rate + score_penalty + dti_penalty + util_penalty +
loan_type_penalty.get(model_payload["loan_type"], 1.8) - stability_discount
return float(np.clip(rate, 6.5, 18.5)), info

def evaluate_application(bundle: Optional[Dict[str, Any]], payload: Dict[str, Any]) ->
Dict[str, Any]:
    prob, prob_info = predict_approval_probability(bundle, payload)
    logic_payload = to_model_payload(payload)
    fsi = calculate_fsi(logic_payload)
    risk = get_risk(prob, fsi)
    decision = final_decision(prob, fsi)
    health = loan_health_score(prob, fsi)
    emi = logic_payload["emi"]
    affordability = check_affordability(emi, logic_payload["annual_income"] / 12)
    rejection = rejection_breakdown(logic_payload, prob, fsi)
    suggestions = combined_suggestions(logic_payload, prob, fsi)
    early_warning = default_early_warning(logic_payload, prob, fsi)
    return {
        "payload": logic_payload,
        "probability": prob,
        "probability_info": prob_info,
        "fsi": fsi,
        "risk": risk,
        "decision": decision,
        "health_score": health,
        "affordability": affordability,
        "emi": emi,
        "rejection_breakdown": rejection,
        "suggestions": suggestions,
        "early_warning": early_warning,
    }

def rejection_breakdown(data: Dict[str, Any], prob: float, fsi: float) -> List[Dict[str,
Any]]:
    raw_scores = {
        "Low credit score": max(0.0, (680 - data["credit_score"]) / 180),
        "High debt-to-income": max(0.0, (data["debt_to_income_ratio"] - 0.35) / 0.35),
        "High EMI burden": max(0.0, (data["emi_to_income_ratio"] - 0.35) / 0.35),
        "High credit utilization": max(0.0, (data["credit_utilization_ratio"] - 0.45) /
0.4),

```

```

        "Recent repayment stress": max(0.0, (70 - data["repayment_history_score"]) / 50),
        "Default probability pressure": max(0.0, (prob - 0.45) / 0.55),
        "Low financial stability": max(0.0, (65 - fsi) / 35),
    }
    total = sum(raw_scores.values())
    if total <= 0:
        return [{"reason": "Profile looks stable", "share": 100.0, "detail": "No major
rejection drivers detected."}]
    rows = []
    for reason, score in raw_scores.items():
        if score <= 0:
            continue
        rows.append(
            {
                "reason": reason,
                "share": round(score / total * 100, 1),
                "detail": reason_detail(reason, data, prob, fsi),
            }
        )
    rows.sort(key=lambda item: item["share"], reverse=True)
    return rows

def reason_detail(reason: str, data: Dict[str, Any], prob: float, fsi: float) -> str:
    mapping = {
        "Low credit score": f"Credit score is {int(data['credit_score'])}, which weakens
pricing and approval confidence.",
        "High debt-to-income": f"Debt-to-income ratio is
{data['debt_to_income_ratio']:.2f}; lenders usually prefer a lower burden.",
        "High EMI burden": f"EMI-to-income ratio is {data['emi_to_income_ratio']:.2f},
showing monthly pressure.",
        "High credit utilization": f"Credit utilization is
{data['credit_utilization_ratio']:.2f}; lower usage improves risk profile.",
        "Recent repayment stress": f"Repayment history score is
{data['repayment_history_score']:.1f}, indicating uneven discipline.",
        "Default probability pressure": f"Predicted default probability is {prob:.1%}.",
        "Low financial stability": f"Financial Stability Index is {fsi:.1f} on a 0-100
scale.",
    }
    return mapping.get(reason, reason)

def combined_suggestions(data: Dict[str, Any], prob: float, fsi: float) -> List[str]:
    suggestions = list(suggest_improvements(data))
    if data["credit_utilization_ratio"] > 0.5:
        suggestions.append("Bring credit utilization below 30% to improve approval
strength.")
    if data["debt_to_income_ratio"] > 0.45:
        suggestions.append("Reduce debt before applying or stretch tenure to ease debt
pressure.")
    if data["emi_to_income_ratio"] > 0.4:
        suggestions.append("Lower loan amount or add a co-applicant to keep EMI
manageable.")
    if prob > 0.55:
        suggestions.append("Apply after 3-6 months of stronger repayment behavior to
improve probability.")
    if fsi < 60:
        suggestions.append("Increase emergency buffer and income stability for a better
financial health score.")
    seen = set()
    unique = []

```

```

for item in suggestions:
    if item not in seen:
        unique.append(item)
        seen.add(item)
return unique

def credit_utilization_analyzer(utilization: float) -> Dict[str, str]:
    if utilization < 0.3:
        level = "Healthy"
        impact = "Low risk impact"
        suggestion = "Current utilization is well controlled."
    elif utilization < 0.6:
        level = "Watch"
        impact = "Moderate risk impact"
        suggestion = "Try to reduce balances before the next billing cycle."
    else:
        level = "High"
        impact = "Strong negative impact"
        suggestion = "High utilization is lifting risk. Pay down revolving debt quickly."
    return {"level": level, "impact": impact, "suggestion": suggestion}

def default_early_warning(data: Dict[str, Any], prob: float, fsi: float) -> Dict[str, str]:
    stress_index = (
        prob * 0.45
        + min(1.0, data["emi_to_income_ratio"]) * 0.25
        + min(1.0, data["debt_to_income_ratio"]) * 0.2
        + data["credit_utilization_ratio"] * 0.1
    )
    if stress_index > 0.7 or fsi < 50:
        level = "High"
        note = "Profile shows elevated future default risk under normal conditions."
    elif stress_index > 0.48 or fsi < 65:
        level = "Moderate"
        note = "Profile is workable but sensitive to income or rate shocks."
    else:
        level = "Low"
        note = "Profile appears resilient with manageable payment stress."
    return {"level": level, "note": note}

def generate_probability_curve(bundle: Optional[Dict[str, Any]], payload: Dict[str, Any]) -
> pd.DataFrame:
    base = dict(payload)
    rows = []
    for credit_score in range(500, 851, 25):
        sim = dict(base)
        sim["credit_score"] = credit_score
        prob, _ = predict_approval_probability(bundle, sim)
        fsi = calculate_fsi(to_model_payload(sim))
        rows.append({"credit_score": credit_score, "approval_chance": round((1 - prob) *
100, 2), "fsi": fsi})
    return pd.DataFrame(rows)

def variable_impact_table(bundle: Optional[Dict[str, Any]], payload: Dict[str, Any]) ->
pd.DataFrame:
    base = evaluate_application(bundle, payload)
    candidates = [
        ("Credit Score +50", {"credit_score": min(850, payload["credit_score"] + 50)}),

```

```

        ("Income +20%", {"annual_income": payload["annual_income"] * 1.2}),
        ("Loan Amount -15%", {"loan_amount": payload["loan_amount"] * 0.85}),
        ("Utilization -15%", {"credit_utilization_ratio": max(0.0,
payload["credit_utilization_ratio"] - 0.15)}),
        ("Tenure +24m", {"loan_term_months": payload["loan_term_months"] + 24}),
    ]
    rows = []
    for label, patch in candidates:
        sim = dict(payload)
        sim.update(patch)
        result = evaluate_application(bundle, sim)
        rows.append(
            {
                "scenario": label,
                "approval_change": round((1 - result["probability"] - (1 -
base["probability"]))) * 100, 2),
                "fsi_change": round(result["fsi"] - base["fsi"], 2),
                "emi_change": round(result["emi"] - base["emi"], 2),
            }
        )
    return pd.DataFrame(rows)

def safe_loan_limit(payload: Dict[str, Any]) -> float:
    monthly_income = payload["annual_income"] / 12
    max_emi = monthly_income * 0.35
    term = max(int(payload["loan_term_months"]), 12)
    rate = max(float(payload["interest_rate"]), 6.5)
    monthly_rate = rate / (12 * 100)
    if monthly_rate == 0:
        base_limit = max_emi * term
    else:
        base_limit = max_emi * (((1 + monthly_rate) ** term - 1) / (monthly_rate * (1 +
monthly_rate) ** term))
    profile_factor = (
        payload["credit_score"] / 850 * 0.35
        + (1 - min(1.0, payload["credit_utilization_ratio"])) * 0.2
        + min(1.0, payload["income_stability_score"] / 100) * 0.25
        + (1 - min(1.0, payload["spending_to_income_ratio"])) * 0.2
    )
    return max(50000.0, base_limit * (0.8 + profile_factor))

def recommend_loans(bundle: Optional[Dict[str, Any]], payload: Dict[str, Any]) ->
pd.DataFrame:
    rows = []
    safe_limit = safe_loan_limit(payload)
    for loan_type, cfg in LOAN_CONFIG.items():
        type_cap = safe_limit * cfg["cap_multiple"]
        candidate_amounts = sorted(
            {
                round(min(payload["loan_amount"], type_cap), -3),
                round(min(payload["loan_amount"] * 0.85, type_cap), -3),
                round(min(payload["loan_amount"] * 0.7, type_cap), -3),
            }
        )
        candidate_amounts = [amount for amount in candidate_amounts if amount >= 50000]
        for term in cfg["terms"]:
            for amount in candidate_amounts:
                sim = dict(payload)
                sim["loan_type"] = loan_type

```

```

sim["loan_term_months"] = term
sim["loan_amount"] = float(amount)
sim["interest_rate"], _ = estimate_interest_rate(bundle, sim)
result = evaluate_application(bundle, sim)
approval = 1 - result["probability"]
utility = approval * 0.55 + (result["health_score"] / 100) * 0.25
utility -= min(1.0, result["payload"]["emi_to_income_ratio"]) * 0.2
rows.append(
    {
        "loan_type": cfg["label"],
        "loan_type_key": loan_type,
        "loan_amount": sim["loan_amount"],
        "loan_term_months": term,
        "interest_rate": sim["interest_rate"],
        "emi": result["emi"],
        "approval_chance": round(approval * 100, 2),
        "risk": result["risk"],
        "decision": result["decision"],
        "health_score": result["health_score"],
        "utility_score": round(utility * 100, 2),
    }
)
rec_df = pd.DataFrame(rows)
rec_df = rec_df.sort_values(["utility_score", "approval_chance", "health_score"],
ascending=False)
return rec_df.head(12).reset_index(drop=True)

def coapplicant_optimization(bundle: Optional[Dict[str, Any]], payload: Dict[str, Any],
co_income: float, co_credit_score: int, debt_share_ratio: float) -> Dict[str, Any]:
    base = evaluate_application(bundle, payload)
    sim = dict(payload)
    sim["annual_income"] = payload["annual_income"] + max(0.0, co_income)
    sim["credit_score"] = int(round((payload["credit_score"] * 0.65) + (co_credit_score *
0.35)))
    sim["total_debt"] = max(0.0, payload["total_debt"] * (1 - debt_share_ratio))
    sim["income_stability_score"] = min(100.0, payload["income_stability_score"] + 8)
    sim["job_stability_score"] = min(100.0, payload["job_stability_score"] + 5)
    upgraded = evaluate_application(bundle, sim)
    delta_approval = round(((1 - upgraded["probability"]) - (1 - base["probability"])) *
100, 2)
    return {
        "base": base,
        "with_coapplicant": upgraded,
        "approval_gain": delta_approval,
        "should_add": delta_approval > 5 or upgraded["decision"] != base["decision"],
    }

def compare_loan_scenarios(bundle: Optional[Dict[str, Any]], scenarios: List[Dict[str,
Any]]) -> pd.DataFrame:
    rows = []
    for item in scenarios:
        label = item["label"]
        payload = dict(item["payload"])
        result = evaluate_application(bundle, payload)
        schedule = amortization_schedule(
            payload["loan_amount"], payload["interest_rate"],
int(payload["loan_term_months"]), extra_payment=0.0
        )
        rows.append(

```

```

        {
            "scenario": label,
            "loan_type": payload["loan_type"].title(),
            "loan_amount": payload["loan_amount"],
            "tenure_months": payload["loan_term_months"],
            "rate": payload["interest_rate"],
            "emi": result["emi"],
            "approval_chance": round((1 - result["probability"]) * 100, 2),
            "total_interest": round(float(schedule["interest"].sum()), 2),
            "decision": result["decision"],
        }
    )
    return pd.DataFrame(rows)

def amortization_schedule(loan_amount: float, annual_rate: float, months: int,
extra_payment: float = 0.0) -> pd.DataFrame:
    r = annual_rate / (12 * 100.0)
    emi = calculate_emi(loan_amount, annual_rate, months)
    balance = float(loan_amount)
    rows = []
    month = 1
    while month <= months and balance > 0:
        interest = balance * r
        principal = min(balance, max(0.0, emi - interest) + extra_payment)
        balance = max(0.0, balance - principal)
        rows.append({"month": month, "interest": interest, "principal": principal,
"balance": balance})
        month += 1
    if balance <= 0:
        break
    return pd.DataFrame(rows)

def prepayment_summary(loan_amount: float, annual_rate: float, months: int, extra_payment:
float) -> Dict[str, float]:
    base = amortization_schedule(loan_amount, annual_rate, months, extra_payment=0.0)
    new = amortization_schedule(loan_amount, annual_rate, months,
extra_payment=extra_payment)
    base_interest = float(base["interest"].sum())
    new_interest = float(new["interest"].sum())
    base_months = int(base["month"].max()) if not base.empty else 0
    new_months = int(new["month"].max()) if not new.empty else 0
    return {
        "interest_saved": round(base_interest - new_interest, 2),
        "months_saved": max(0, base_months - new_months),
        "base_interest": round(base_interest, 2),
        "new_interest": round(new_interest, 2),
    }

def financial_goal_plan(_bundle: Optional[Dict[str, Any]], payload: Dict[str, Any],
goal_type: str, down_payment: float, years_to_goal: int) -> Dict[str, Any]:
    horizon_factor = max(1.0, years_to_goal / 2)
    safe_limit = safe_loan_limit(payload) * min(1.2, 0.85 + 0.08 * horizon_factor)
    goal_caps = {"Buy House": 1.15, "Buy Car": 0.65, "Higher Education": 0.55, "Business
Expansion": 0.7}
    target_loan = safe_limit * goal_caps.get(goal_type, 0.7)
    target_loan = max(50000.0, target_loan - down_payment * 0.2)
    monthly_income = payload["annual_income"] / 12
    comfortable_emi = monthly_income * 0.32

```

```

return {
    "goal_type": goal_type,
    "safe_loan_amount": round(target_loan, 2),
    "comfortable_emi": round(comfortable_emi, 2),
    "years_to_goal": years_to_goal,
    "down_payment": down_payment,
    "note": f"You can target around {format_currency(target_loan)} safely for
{goal_type.lower()}.",
}

def applicant_benchmark(payload: Dict[str, Any], raw_df: pd.DataFrame) -> pd.DataFrame:
    metrics = [
        ("Credit Score", payload["credit_score"], float(raw_df["credit_score"].median()),
"Higher is better"),
        ("Annual Income", payload["annual_income"],
float(raw_df["annual_income"].median()), "Higher is better"),
        ("Debt To Income", payload["total_debt"] / max(payload["annual_income"], 1.0),
float(raw_df["debt_to_income_ratio"].median()), "Lower is better"),
        ("Credit Utilization", payload["credit_utilization_ratio"],
float(raw_df["credit_utilization_ratio"].median()), "Lower is better"),
        ("Repayment Score", payload["repayment_history_score"],
float(raw_df["repayment_history_score"].median()), "Higher is better"),
    ]
    rows = []
    for label, applicant_value, portfolio_value, rule in metrics:
        if rule == "Higher is better":
            status = "Stronger" if applicant_value >= portfolio_value else "Needs
improvement"
        else:
            status = "Stronger" if applicant_value <= portfolio_value else "Needs
improvement"
        rows.append(
            {
                "metric": label,
                "applicant_value": round(float(applicant_value), 2),
                "portfolio_median": round(float(portfolio_value), 2),
                "status": status,
            }
        )
    return pd.DataFrame(rows)

def risk_heatmap_frame(df: pd.DataFrame) -> pd.DataFrame:
    work = df.copy()
    work["credit_band"] = pd.cut(work["credit_score"], bins=[300, 600, 680, 750, 850],
labels=["300-600", "600-680", "680-750", "750-850"], include_lowest=True)
    work["dti_band"] = pd.cut(work["debt_to_income_ratio"], bins=[0, 0.5, 1, 2, 10],
labels=["0-0.5", "0.5-1", "1-2", "2+"], include_lowest=True)
    pivot = work.pivot_table(index="dti_band", columns="credit_band", values="loan_status",
aggfunc="mean", observed=False)
    return pivot.fillna(0.0)

def approval_trend_frame(df: pd.DataFrame) -> pd.DataFrame:
    work = df.copy()
    work["approval_flag"] = 1 - work["loan_status"]
    return work.groupby("loan_type", as_index=False).agg(
        approval_rate=("approval_flag", "mean"),
        avg_interest_rate=("interest_rate", "mean"),
        avg_credit_score=("credit_score", "mean"),

```



```

)

def portfolio_summary(df: pd.DataFrame) -> Dict[str, Any]:
    work = df.copy()
    work["approval_flag"] = 1 - work["loan_status"]
    work["risk_flag"] = np.where(
        (work["credit_score"] < 650)
        | (work["credit_utilization_ratio"] > 0.6)
        | (work["debt_to_income_ratio"] > 0.7),
        "High",
        np.where((work["credit_score"] < 720) | (work["debt_to_income_ratio"] > 0.45),
        "Medium", "Low"),
    )
    total_approved = int(work["approval_flag"].sum())
    high_risk_share = float((work["risk_flag"] == "High").mean())
    return {
        "total_approved": total_approved,
        "portfolio_size": int(work.shape[0]),
        "approval_rate": float(work["approval_flag"].mean()),
        "high_risk_share": high_risk_share,
        "warning": "Warning: high-risk mix is elevated." if high_risk_share > 0.3 else
        "Portfolio mix is within a healthy range.",
    }

def save_profile_event(event: Dict[str, Any]) -> None:
    HISTORY_DIR.mkdir(parents=True, exist_ok=True)
    history = load_profile_history()
    history.append(event)
    HISTORY_FILE.write_text(json.dumps(history, indent=2))

def load_profile_history() -> List[Dict[str, Any]]:
    if not HISTORY_FILE.exists():
        return []
    try:
        return json.loads(HISTORY_FILE.read_text())
    except Exception:
        return []

def build_history_frame() -> pd.DataFrame:
    history = load_profile_history()
    if not history:
        return pd.DataFrame(columns=["timestamp", "name", "loan_type", "loan_amount",
        "approval_chance", "decision", "risk"])
    rows = []
    for item in history:
        rows.append(
            {
                "timestamp": item.get("timestamp"),
                "name": item.get("name", "Applicant"),
                "loan_type": item.get("loan_type", "").title(),
                "loan_amount": item.get("loan_amount"),
                "approval_chance": item.get("approval_chance"),
                "decision": item.get("decision"),
                "risk": item.get("risk"),
            }
        )
    return pd.DataFrame(rows)

```

```

def assistant_action_plan(payload: Dict[str, Any], bundle: Optional[Dict[str, Any]] = None)
-> List[str]:
    result = evaluate_application(bundle, payload)
    actions = list(result["suggestions"][:4])
    if payload["credit_score"] < 700:
        actions.append("Focus on improving credit score before asking for a larger ticket
size.")
    if payload["credit_utilization_ratio"] > 0.5:
        actions.append("Pay down revolving credit first because utilization is hurting
approval quality.")
    if result["affordability"] != "Affordable":
        actions.append("Reduce EMI pressure by extending tenure or lowering requested
amount.")
    if result["decision"] != "APPROVED":
        actions.append("Reapply with a stronger profile snapshot after the next repayment
cycle.")
    seen = set()
    unique = []
    for item in actions:
        if item not in seen:
            unique.append(item)
            seen.add(item)
    return unique[:5]

def assistant_response(choice: str, payload: Optional[Dict[str, Any]] = None, bundle:
Optional[Dict[str, Any]] = None) -> Dict[str, Any]:
    if payload is None:
        return {
            "title": "Assistant Ready",
            "answer": "Open Smart Application, enter a borrower profile, and I will
generate personalized guidance.",
            "bullets": [
                "I can explain eligibility.",
                "I can suggest better loan structures.",
                "I can show EMI and prepayment guidance.",
            ],
            "highlights": [],
        }

    result = evaluate_application(bundle, payload)
    approval_chance = (1 - result["probability"]) * 100
    recs = recommend_loans(bundle, payload)
    top = recs.iloc[0]

    if choice == "Check EMI":
        return {
            "title": "EMI Guidance",
            "answer": f"Estimated EMI is {format_currency(result['emi'])} per month and the
profile is currently marked as {result['affordability'].lower()}.",
            "bullets": [
                f"Current tenure is {payload['loan_term_months']} months.",
                f"Monthly burden ratio is {result['payload']['emi_to_income_ratio']:.2f}.",
                "Use the EMI and Prepayment tab to test lower tenure or faster closure.",
            ],
            "highlights": [
                ("EMI", format_currency(result["emi"])),
                ("Affordability", result["affordability"]),
            ],
        }

```

```

    }
    if choice == "Loan eligibility help":
        return {
            "title": "Eligibility Guidance",
            "answer": f"Current decision trend is {result['decision']} with
{approval_chance:.1f}% approval chance and {result['risk']} classification.",
            "bullets": [
                f"FSI is {result['fsi']:.1f}, which reflects financial stability.",
                f"Early warning level is {result['early_warning']['level']}.",
                result["rejection_breakdown"][0]["detail"],
            ],
            "highlights": [
                ("Approval", f"{approval_chance:.1f}%"),
                ("FSI", f"{result['fsi']:.1f}"),
                ("Decision", result["decision"]),
            ],
        }
    if choice == "Improve approval chances":
        return {
            "title": "Approval Improvement Plan",
            "answer": "These are the strongest next steps to improve approval quality for
the current profile.",
            "bullets": assistant_action_plan(payload, bundle),
            "highlights": [
                ("Top Risk", result["rejection_breakdown"][0]["reason"]),
                ("Health Score", f"{result['health_score']:.1f}"),
            ],
        }
    if choice == "Best loan type":
        return {
            "title": "Best Loan Match",
            "answer": f"Best current fit is a {top['loan_type']} loan with
{int(top['loan_term_months'])} months tenure and
{format_currency(float(top['loan_amount']))} amount.",
            "bullets": [
                f"Estimated approval chance is {float(top['approval_chance']):.1f}%.",
                f"Expected EMI is {format_currency(float(top['emi']))}.",
                f"Recommended decision outlook is {top['decision']}.",
            ],
            "highlights": [
                ("Loan Type", top["loan_type"]),
                ("Tenure", f"{int(top['loan_term_months'])}m"),
                ("Amount", format_currency(float(top["loan_amount"]))),
            ],
        }
    if choice == "Can I lower my EMI":
        lower_amount = max(50000.0, payload["loan_amount"] * 0.85)
        lower_tenure = payload["loan_term_months"] + 24
        new_emi = calculate_emi(lower_amount, payload["interest_rate"], lower_tenure)
        return {
            "title": "EMI Reduction Strategy",
            "answer": f"If you reduce the loan amount to {format_currency(lower_amount)}
and extend tenure to {lower_tenure} months, EMI can move near {format_currency(new_emi)}.",
            "bullets": [
                "Lower amount has the fastest EMI impact.",
                "Longer tenure reduces EMI but may increase total interest.",
                "Use prepayment later to recover some interest cost.",
            ],
            "highlights": [
                ("Current EMI", format_currency(result["emi"])),
                ("Target EMI", format_currency(new_emi)),
            ],
        }

```

```

    ],
    }
    if choice == "Should I add co-applicant":
        return {
            "title": "Co-Applicant Advice",
            "answer": "A co-applicant is most useful when income support and risk balancing
are both needed.",
            "bullets": [
                "Add a co-applicant if EMI burden feels high relative to income.",
                "It helps more when the co-applicant has stronger credit than the primary
borrower.",
                "Use Recommendation Lab to test income and credit score improvements
live.",
            ],
            "highlights": [
                ("Current EMI Burden", f"{result['payload']['emi_to_income_ratio']:.2f}"),
                ("Current Decision", result["decision"]),
            ],
        }
    if choice == "Rate negotiation tips":
        est_rate, _ = estimate_interest_rate(bundle, payload)
        return {
            "title": "Rate Negotiation Tips",
            "answer": f"Your smart estimated rate is around {est_rate:.2f}%, which you can
use as a benchmark while negotiating.",
            "bullets": [
                "Negotiate after improving credit utilization and repayment behavior.",
                "A stronger co-applicant can help justify better pricing.",
                "Lower requested amount and stronger down payment also support a better
offer.",
            ],
            "highlights": [
                ("Estimated Rate", f"{est_rate:.2f}%"),
                ("Credit Score", f"{payload['credit_score']}"),
            ],
        }

    return {
        "title": "Assistant Ready",
        "answer": "Choose an action and I will guide you using the current borrower
profile.",
        "bullets": [],
        "highlights": [],
    }

def stress_delta_summary(base_prob: float, stress_prob: float) -> Dict[str, Any]:
    return {
        "base_approval": round((1 - base_prob) * 100, 2),
        "stress_approval": round((1 - stress_prob) * 100, 2),
        "impact": stress_impact(base_prob, stress_prob),
    }

```

4. Src/train_advanced_models.py

```

from pathlib import Path

import joblib
import pandas as pd

```

```

from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.metrics import accuracy_score, mean_absolute_error, r2_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

```

```

BASE_DIR = Path(__file__).resolve().parent.parent
RAW_DATA = BASE_DIR / "data" / "raw" / "creditpilot_dataset.csv"
MODEL_PATH = BASE_DIR / "models" / "advanced_bundle.pkl"

```

```

def build_preprocessor(df: pd.DataFrame) -> ColumnTransformer:
    categorical = ["employment_type", "loan_type"]
    numeric = [col for col in df.columns if col not in categorical]
    return ColumnTransformer(
        transformers=[
            ("num", StandardScaler(), numeric),
            ("cat", OneHotEncoder(handle_unknown="ignore"), categorical),
        ]
    )

```

```

def train_bundle() -> dict:
    df = pd.read_csv(RAW_DATA)

    approval_features = [
        "age",
        "employment_type",
        "employment_length_years",
        "annual_income",
        "income_stability_score",
        "loan_amount",
        "loan_term_months",
        "interest_rate",
        "loan_type",
        "emi",
        "total_existing_loans",
        "total_debt",
        "debt_to_income_ratio",
        "credit_score",
        "credit_utilization_ratio",
        "emi_to_income_ratio",
        "loan_to_income_ratio",
        "num_credit_inquiries",
        "late_payment_count",
        "default_history",
        "repayment_history_score",
        "credit_history_length_years",
        "job_stability_score",
        "spending_to_income_ratio",
    ]
    X_cls = df[approval_features]
    y_cls = df["loan_status"]
    X_train, X_test, y_train, y_test = train_test_split(
        X_cls, y_cls, test_size=0.2, random_state=42, stratify=y_cls
    )
    cls_pipe = Pipeline(
        steps=[
            ("prep", build_preprocessor(X_cls)),

```

```

        ("model", RandomForestClassifier(n_estimators=240, max_depth=10,
min_samples_leaf=4, random_state=42)),
    ]
)
cls_pipe.fit(X_train, y_train)
cls_pred = cls_pipe.predict(X_test)
cls_prob = cls_pipe.predict_proba(X_test)[:, 1]

interest_features = [
    "age",
    "employment_type",
    "employment_length_years",
    "annual_income",
    "income_stability_score",
    "loan_amount",
    "loan_term_months",
    "loan_type",
    "emi",
    "total_existing_loans",
    "total_debt",
    "debt_to_income_ratio",
    "credit_score",
    "credit_utilization_ratio",
    "emi_to_income_ratio",
    "loan_to_income_ratio",
    "num_credit_inquiries",
    "late_payment_count",
    "default_history",
    "repayment_history_score",
    "credit_history_length_years",
    "job_stability_score",
    "spending_to_income_ratio",
    "loan_status",
]
X_reg = df[interest_features]
y_reg = df["interest_rate"]
Xr_train, Xr_test, yr_train, yr_test = train_test_split(X_reg, y_reg, test_size=0.2,
random_state=42)
reg_pipe = Pipeline(
    steps=[
        ("prep", build_preprocessor(X_reg)),
        ("model", RandomForestRegressor(n_estimators=220, max_depth=12,
min_samples_leaf=3, random_state=42)),
    ]
)
reg_pipe.fit(Xr_train, yr_train)
reg_pred = reg_pipe.predict(Xr_test)

bundle = {
    "approval_model": cls_pipe,
    "interest_model": reg_pipe,
    "metrics": {
        "approval": {
            "accuracy": float(accuracy_score(y_test, cls_pred)),
            "roc_auc": float(roc_auc_score(y_test, cls_prob)),
        },
        "interest": {
            "mae": float(mean_absolute_error(yr_test, reg_pred)),
            "r2": float(r2_score(yr_test, reg_pred)),
        },
    },
},

```

```

        "feature_columns": {
            "approval": approval_features,
            "interest": interest_features,
        },
    }
    MODEL_PATH.parent.mkdir(parents=True, exist_ok=True)
    joblib.dump(bundle, MODEL_PATH)
    return bundle

if __name__ == "__main__":
    trained = train_bundle()
    print("Saved:", MODEL_PATH)
    print("Approval metrics:", trained["metrics"]["approval"])
    print("Interest metrics:", trained["metrics"]["interest"])

```

B. DATA DICTIONARY

- age: Age of the applicant.
- annual_income: Total yearly earnings.
- dti: Debt-to-income ratio (Total Monthly Debt / Monthly Income).
- fsi: Financial Stability Index (calculated composite).
- utilization: Credit card balance / total credit limit.
- loan_to_income: Ratio of requested loan amount to annual income.

C. USER MANUAL AND DEPLOYMENT GUIDE

1. Install Python 3.11.
2. Run `pip install -r requirements.txt`.
3. Place `advanced\_bundle.pkl` in the `/models` folder.
4. Execute `streamlit run src/app.py`.
5. Access the dashboard via `http://localhost:8501`.

D. PROJECT SNAPSHOTS

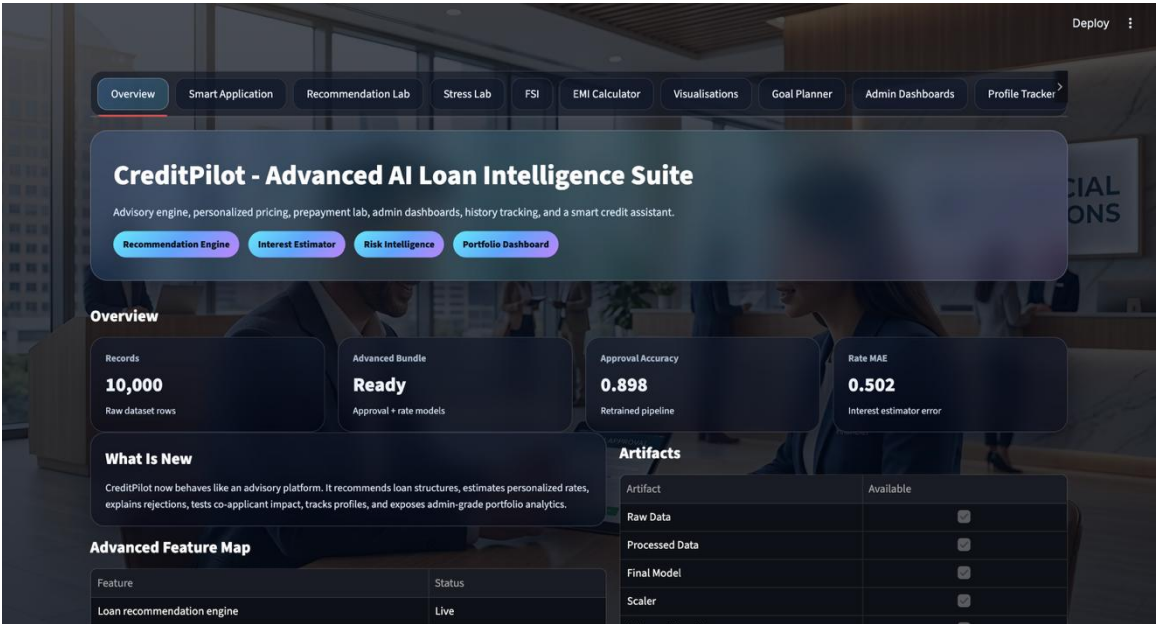


Figure 3: Dashboard Overview

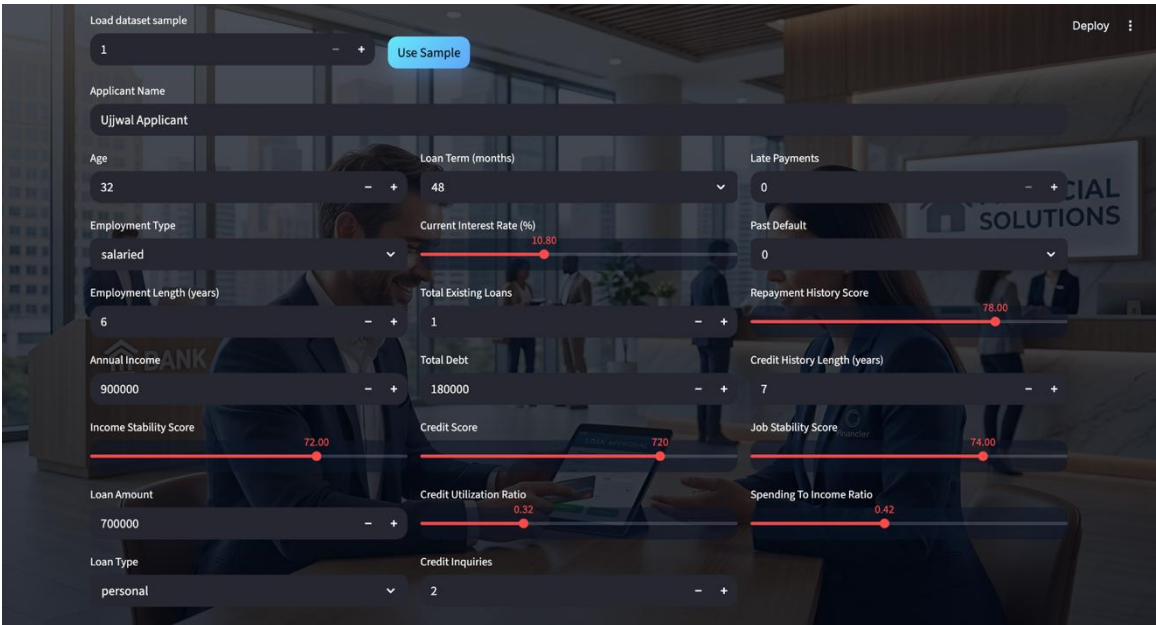


Figure 4: Sample Values



Figure 5: Loan Analysis

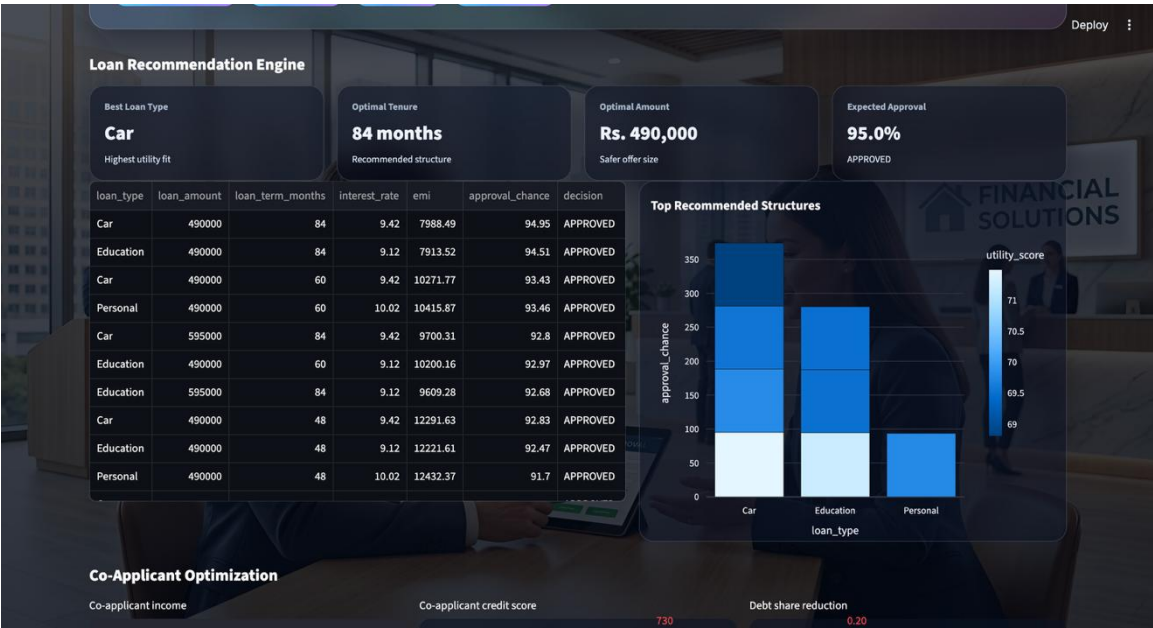


Figure 6: Loan Recommendation Engine

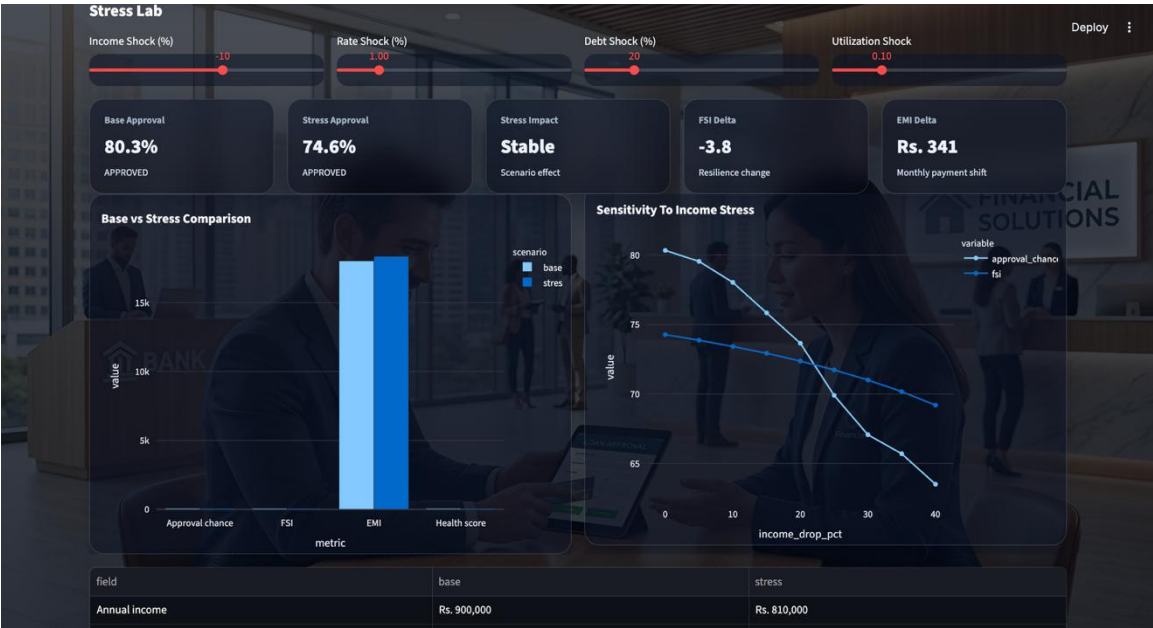


Figure 7: Stress Lab



Figure 8: Component Breakdown

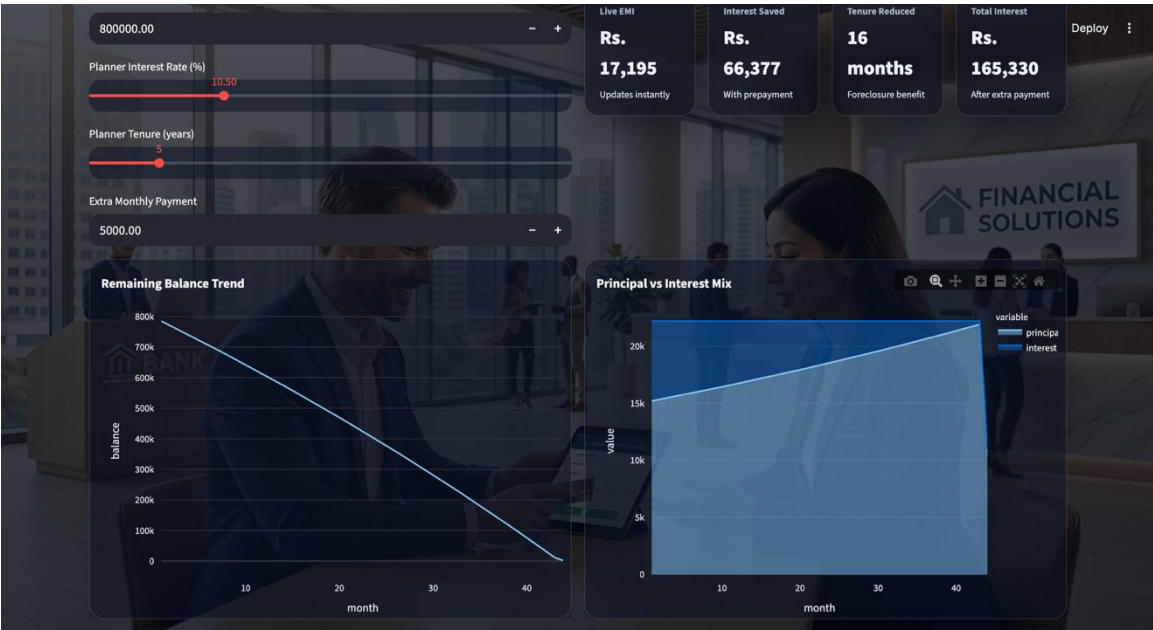


Figure 9: Loan planning



Figure 10: Data Visualization

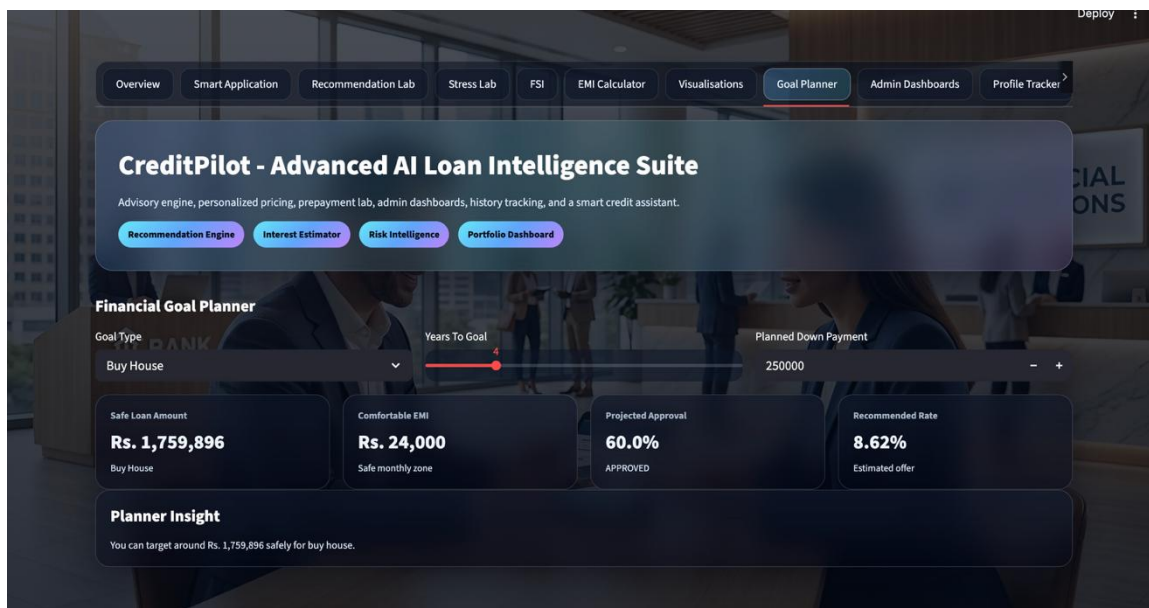


Figure 11: Goal Planner

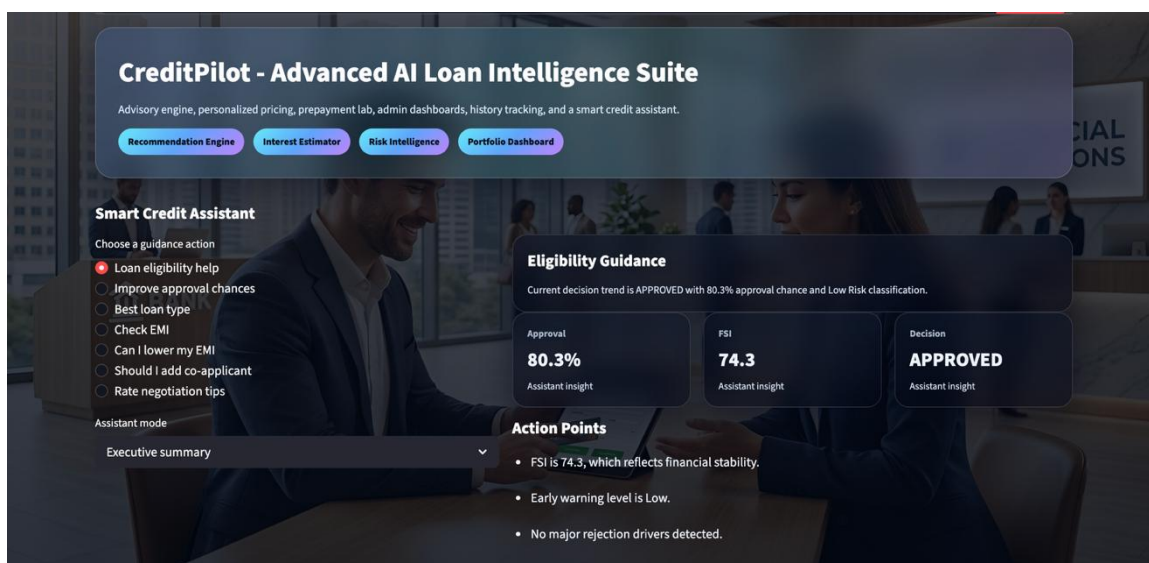


Figure 12: Smart Credit Assistant

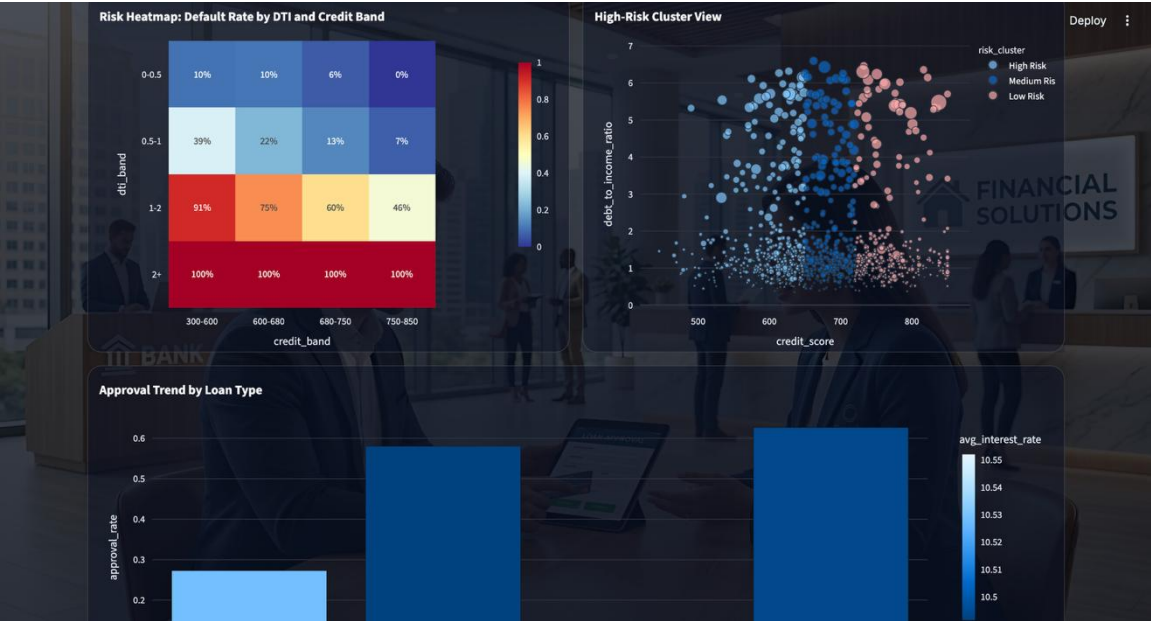


Figure 13: Data Visualization- II